

Proyecto Integrador: Clasificación Supervisada

Alex, Martina, Valentina, Sebastián e Izaak

2026-08-07

Índice

Objetivo del proyecto	5
Instalación y carga de librerías	5
Carga y limpieza de datos	6
Análisis exploratorio (EDA)	9
Partición de datos (Train / Test) de forma estratificada	11
Preparación de variables para los modelos	12
Estandarización	14
Configuración de validación cruzada	14
Modelo baseline	17
=====	
REGRESIÓN LOGÍSTICA (CORREGIDA: preProcess dentro de cada fold)	18
=====	
=====	
EVALUACIÓN DE REGRESIÓN LOGÍSTICA EN TEST	19
=====	
=====	
K-NEAREST NEIGHBORS (KNN)	21
=====	

EVALUACIÓN DE KNN EN TEST	23
---------------------------	----

SVM CON KERNEL LINEAL (CON PROCESAMIENTO EN PARALELO)	25
---	----

EVALUACIÓN DE SVM LINEAL EN TEST	27
----------------------------------	----

SVM CON KERNEL RBF (CON PROCESAMIENTO EN PARALELO)	29
--	----

EVALUACIÓN DE SVM RBF EN TEST	31
-------------------------------	----

ÁRBOL DE CLASIFICACIÓN	33
------------------------	----

VISUALIZACIÓN E IMPORTANCIA DE VARIABLES	34
--	----

EVALUACIÓN DEL ÁRBOL EN TEST	36
------------------------------	----

RANDOM FOREST	37
---------------	----

EVALUACIÓN DE RANDOM FOREST EN TEST	40
-------------------------------------	----

GRADIENT BOOSTING	41
-------------------	----

EVALUACIÓN DE GRADIENT BOOSTING EN TEST	43
---	----

COMPARACIÓN FORMAL DE VALIDACIÓN CRUZADA (resamples)	45
--	----

PR-AUC Y LOG-LOSS EN VALIDACIÓN CRUZADA (SIN USAR TEST)	48
---	----

CALIBRACIÓN DE PROBABILIDADES (SIN USAR TEST)	48
---	----

UMBRAL ÓPTIMO SEGÚN ÍNDICE DE YAUDEN (EN VALIDACIÓN, NO EN TEST)	50
--	----

DEFINICION DE COSTOS Y FUNCION AUXILIAR (se definen aca, ANTES de usarse	51
en el analisis de sensibilidad y en la comparacion Youden vs. Costos)	51

SENSIBILIDAD DEL UMBRAL DE COSTOS A LOS SUPUESTOS DE CF_N/CF_P	52
--	----

COMPARACIÓN: UMBRAL DE YODEN vs. UMBRAL DE COSTOS (EN VALIDACIÓN)	53
---	----

COMPARACIÓN: SISTEMA FINAL vs. BASELINE (EN TEST)	57
---	----

INTERPRETACIÓN: IMPORTANCIA DE VARIABLES EN EL MODELO FINAL (BOOSTING)	57
--	----

CARACTERIZACIÓN DE FALSOS POSITIVOS Y FALSOS NEGATIVOS	58
--	----

DEFINICIÓN DE COSTOS (se usa desde acá en adelante, incluida la tabla comparativa de la Sección 9 y el umbral de costos de la Sección 11)	60
---	----

Objetivo del proyecto

El objetivo de este proyecto es predecir si un cliente de un banco aceptará o no un depósito a plazo (variable `y`, con clases "no" y "yes", siendo "yes" la clase positiva), usando el dataset `bank-additional-full.csv`.

Se elimina la variable `duration` (duración de la llamada telefónica) por tratarse de una variable con **Data Leakage**: no se conoce antes de realizar la llamada, que es el momento en que efectivamente se debe decidir a quién contactar.

Instalación y carga de librerías

```
# Paquetes que necesitamos para todo el proyecto (limpieza, partición,
# validación cruzada, modelos y métricas):
paquetes <- c(
  "tidyverse", # manipulación de datos y gráficos
  "caret",     # partición, validación cruzada y modelos
  "janitor",   # limpieza de nombres y tablas
  "e1071",     # SVM y otras funciones
  "kernlab",   # SVM con kernel (usado por caret)
  "rpart",     # árboles de clasificación
  "rpart.plot", # visualización de árboles
  "pROC",      # curva ROC y AUC
  "scales",    # formato de porcentajes
  "knitr",     # tablas en Markdown
  "ROSE",      # sampling alternativo y utilidades de balanceo
  "PRROC",     # curva Precision-Recall (PR-AUC)
  "MLmetrics", # métricas adicionales, como LogLoss
  "broom"      # para extraer coeficientes de forma ordenada
)

# Instalamos solo los que faltan (se ejecuta una sola vez por máquina)
nuevos <- paquetes[!(paquetes %in% installed.packages()[, "Package"])]
if (length(nuevos) > 0) {
  install.packages(nuevos)
}

# Cargamos todas las librerías. invisible() evita que se imprima en pantalla
# la lista de TRUE/FALSE que devuelve lapply() al cargar cada una.
invisible(lapply(paquetes, library, character.only = TRUE))
```

Carga y limpieza de datos

```
# Nombre del archivo que debe estar en la MISMA carpeta que este .Rmd.
# Al hacer Knit, R Markdown usa como carpeta de trabajo la carpeta donde
# está guardado el .Rmd (no la carpeta donde tengas la consola abierta),
# por eso es clave que el CSV esté justo ahí.
ruta <- "bank-additional-full.csv"

# file.exists() revisa si existe un archivo con ese nombre en la carpeta
# de trabajo actual, y devuelve TRUE o FALSE (no lo abre, solo verifica).
# Si no existe, stop() detiene la ejecución inmediatamente y muestra un
# mensaje claro, en vez de dejar que read.csv2() falle más abajo con un
# error genérico de "no se puede abrir la conexión".
if (!file.exists(ruta)) {
  stop(
    "No se encontró ", ruta, ". ",
    "Debe estar en la misma carpeta que este archivo .Rmd."
  )
}

# read.csv2() se usa porque el archivo viene separado por punto y coma (";"),
# como es típico en datasets europeos (este dataset es de un banco portugués).
datos <- read.csv2(ruta, header = TRUE)

head(datos)
```

```
#>   age      job marital  education default housing loan  contact month
#> 1  56 housemaid married   basic.4y      no      no  no telephone  may
#> 2  57 services married high.school unknown      no      no telephone  may
#> 3  37 services married high.school      no     yes  no telephone  may
#> 4  40 admin. married   basic.6y      no      no  no telephone  may
#> 5  56 services married high.school      no      no yes telephone  may
#> 6  45 services married   basic.9y unknown      no      no telephone  may
#>  day_of_week duration campaign pdays previous  poutcome emp.var.rate
#> 1      mon       261         1    999         0 nonexistent         1.1
#> 2      mon       149         1    999         0 nonexistent         1.1
#> 3      mon       226         1    999         0 nonexistent         1.1
#> 4      mon       151         1    999         0 nonexistent         1.1
#> 5      mon       307         1    999         0 nonexistent         1.1
#> 6      mon       198         1    999         0 nonexistent         1.1
#>  cons.price.idx cons.conf.idx euribor3m nr.employed  y
#> 1      93.994      -36.4      4.857      5191 no
#> 2      93.994      -36.4      4.857      5191 no
#> 3      93.994      -36.4      4.857      5191 no
#> 4      93.994      -36.4      4.857      5191 no
#> 5      93.994      -36.4      4.857      5191 no
#> 6      93.994      -36.4      4.857      5191 no
```

```
str(datos)
```

```
#> 'data.frame':   41188 obs. of  21 variables:
#>  $ age          : int  56 57 37 40 56 45 59 41 24 25 ...
```

```
#> $ job      : chr "housemaid" "services" "services" "admin." ...
#> $ marital   : chr "married" "married" "married" "married" ...
#> $ education : chr "basic.4y" "high.school" "high.school" "basic.6y" ...
#> $ default   : chr "no" "unknown" "no" "no" ...
#> $ housing    : chr "no" "no" "yes" "no" ...
#> $ loan       : chr "no" "no" "no" "no" ...
#> $ contact    : chr "telephone" "telephone" "telephone" "telephone" ...
#> $ month      : chr "may" "may" "may" "may" ...
#> $ day_of_week : chr "mon" "mon" "mon" "mon" ...
#> $ duration   : int 261 149 226 151 307 198 139 217 380 50 ...
#> $ campaign   : int 1 1 1 1 1 1 1 1 1 1 ...
#> $ pdays     : int 999 999 999 999 999 999 999 999 999 999 ...
#> $ previous   : int 0 0 0 0 0 0 0 0 0 0 ...
#> $ poutcome   : chr "nonexistent" "nonexistent" "nonexistent" "nonexistent" ...
#> $ emp.var.rate : chr "1.1" "1.1" "1.1" "1.1" ...
#> $ cons.price.idx : chr "93.994" "93.994" "93.994" "93.994" ...
#> $ cons.conf.idx : chr "-36.4" "-36.4" "-36.4" "-36.4" ...
#> $ euribor3m    : chr "4.857" "4.857" "4.857" "4.857" ...
#> $ nr.employed  : chr "5191" "5191" "5191" "5191" ...
#> $ y            : chr "no" "no" "no" "no" ...
```

```
# Valores faltantes:
colSums(is.na(datos))      # No hay valores NA en este dataset
```

```
#>      age      job      marital      education      default
#>      0        0        0          0              0
#>      housing    loan      contact      month      day_of_week
#>      0        0        0          0              0
#>      duration    campaign      pdays      previous      poutcome
#>      0        0        0          0              0
#>      emp.var.rate cons.price.idx cons.conf.idx      euribor3m      nr.employed
#>      0        0        0          0              0
#>      y
#>      0
```

```
# Valores "unknown" por columna:
sapply(datos, function(x) sum(x == "unknown"))
```

```
#>      age      job      marital      education      default
#>      0      330        80      1731      8597
#>      housing    loan      contact      month      day_of_week
#>      990      990        0          0          0
#>      duration    campaign      pdays      previous      poutcome
#>      0        0        0          0          0
#>      emp.var.rate cons.price.idx cons.conf.idx      euribor3m      nr.employed
#>      0        0        0          0          0
#>      y
#>      0
```

```
# Se concentran en "default", "education", "housing" y "loan". Se dejan como
# una categoría más, ya que "unknown" puede ser información relevante en sí
# misma (ej: un cliente que no declara si tiene crédito).
```

```
# Duplicados:  
sum(duplicated(datos))
```

```
#> [1] 12
```

```
datos <- datos[!duplicated(datos), ]  
sum(duplicated(datos)) # Verificamos que ya no queden
```

```
#> [1] 0
```

```
# Rangos razonables:  
range(datos$age)
```

```
#> [1] 17 98
```

```
range(datos$campaign)
```

```
#> [1] 1 56
```

```
range(datos$pdays) # 999 significa "nunca antes contactado"
```

```
#> [1] 0 999
```

```
# Consistencia y prevalencia del target:  
table(datos$y)
```

```
#>  
#> no yes  
#> 36537 4639
```

```
prop.table(table(datos$y)) # ~88.7% "no" vs ~11.3% "yes": desbalanceado
```

```
#>  
#> no yes  
#> 0.8873 0.1127
```

```
# Algunas columnas numéricas con decimales quedaron como texto (chr) al usar  
# read.csv2(). Las convertimos explícitamente a numérico:  
vars_numeric <- c("emp.var.rate", "cons.price.idx", "cons.conf.idx",  
                 "euribor3m", "nr.employed")  
  
datos[vars_numeric] <- lapply(datos[vars_numeric],  
                             function(x) as.numeric(as.character(x)))  
  
str(datos[vars_numeric])
```

```
#> 'data.frame': 41176 obs. of 5 variables:
#> $ emp.var.rate : num 1.1 1.1 1.1 1.1 1.1 1.1 1.1 1.1 1.1 1.1 ...
#> $ cons.price.idx: num 94 94 94 94 94 ...
#> $ cons.conf.idx : num -36.4 -36.4 -36.4 -36.4 -36.4 -36.4 -36.4 -36.4 -36.4 -36.4 ...
#> $ euribor3m : num 4.86 4.86 4.86 4.86 4.86 ...
#> $ nr.employed : num 5191 5191 5191 5191 5191 ...
```

```
# Eliminamos "duration" por Data Leakage (ver Objetivo del proyecto):
datos <- datos %>% select(-duration)

# Convertimos "y" a factor, fijando explícitamente "no" como clase negativa
# y "yes" como clase positiva:
datos$y <- factor(datos$y, levels = c("no", "yes"))

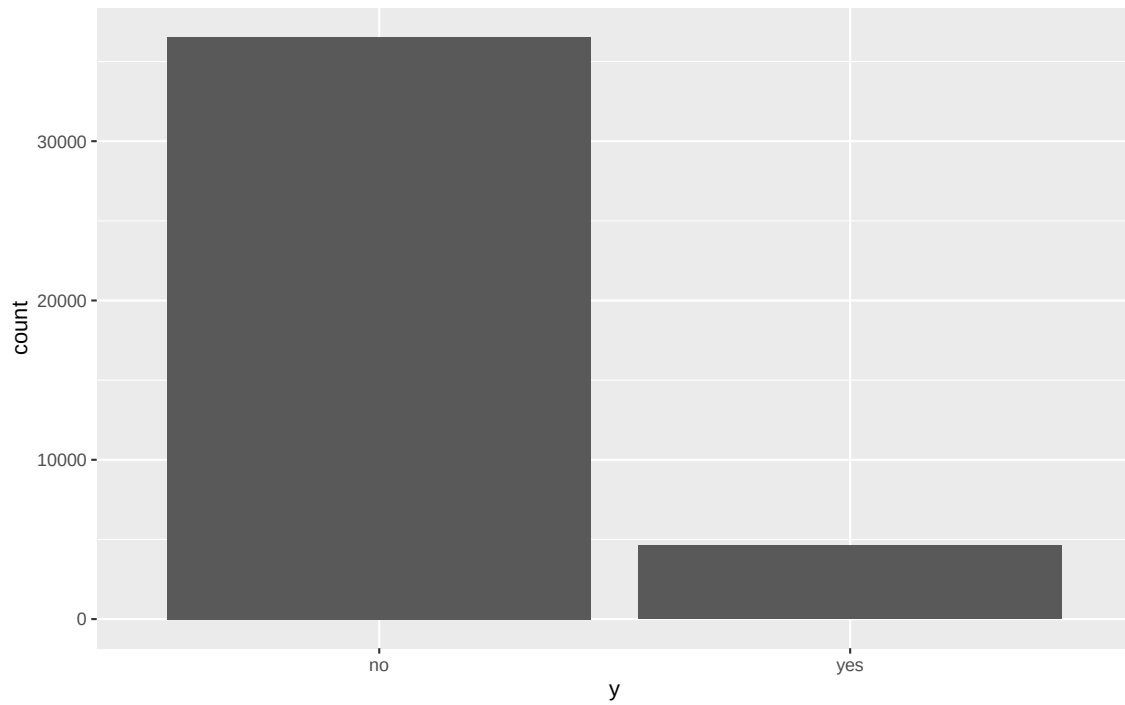
str(datos)
```

```
#> 'data.frame': 41176 obs. of 20 variables:
#> $ age : int 56 57 37 40 56 45 59 41 24 25 ...
#> $ job : chr "housemaid" "services" "services" "admin." ...
#> $ marital : chr "married" "married" "married" "married" ...
#> $ education : chr "basic.4y" "high.school" "high.school" "basic.6y" ...
#> $ default : chr "no" "unknown" "no" "no" ...
#> $ housing : chr "no" "no" "yes" "no" ...
#> $ loan : chr "no" "no" "no" "no" ...
#> $ contact : chr "telephone" "telephone" "telephone" "telephone" ...
#> $ month : chr "may" "may" "may" "may" ...
#> $ day_of_week : chr "mon" "mon" "mon" "mon" ...
#> $ campaign : int 1 1 1 1 1 1 1 1 1 1 ...
#> $ pdays : int 999 999 999 999 999 999 999 999 999 999 ...
#> $ previous : int 0 0 0 0 0 0 0 0 0 0 ...
#> $ poutcome : chr "nonexistent" "nonexistent" "nonexistent" "nonexistent" ...
#> $ emp.var.rate : num 1.1 1.1 1.1 1.1 1.1 1.1 1.1 1.1 1.1 1.1 ...
#> $ cons.price.idx: num 94 94 94 94 94 ...
#> $ cons.conf.idx : num -36.4 -36.4 -36.4 -36.4 -36.4 -36.4 -36.4 -36.4 -36.4 -36.4 ...
#> $ euribor3m : num 4.86 4.86 4.86 4.86 4.86 ...
#> $ nr.employed : num 5191 5191 5191 5191 5191 ...
#> $ y : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...
```

Análisis del resultado. El dataset queda sin duplicados, con las variables numéricas correctamente tipeadas, sin duration, y con y como factor de dos niveles listo para clasificación binaria.

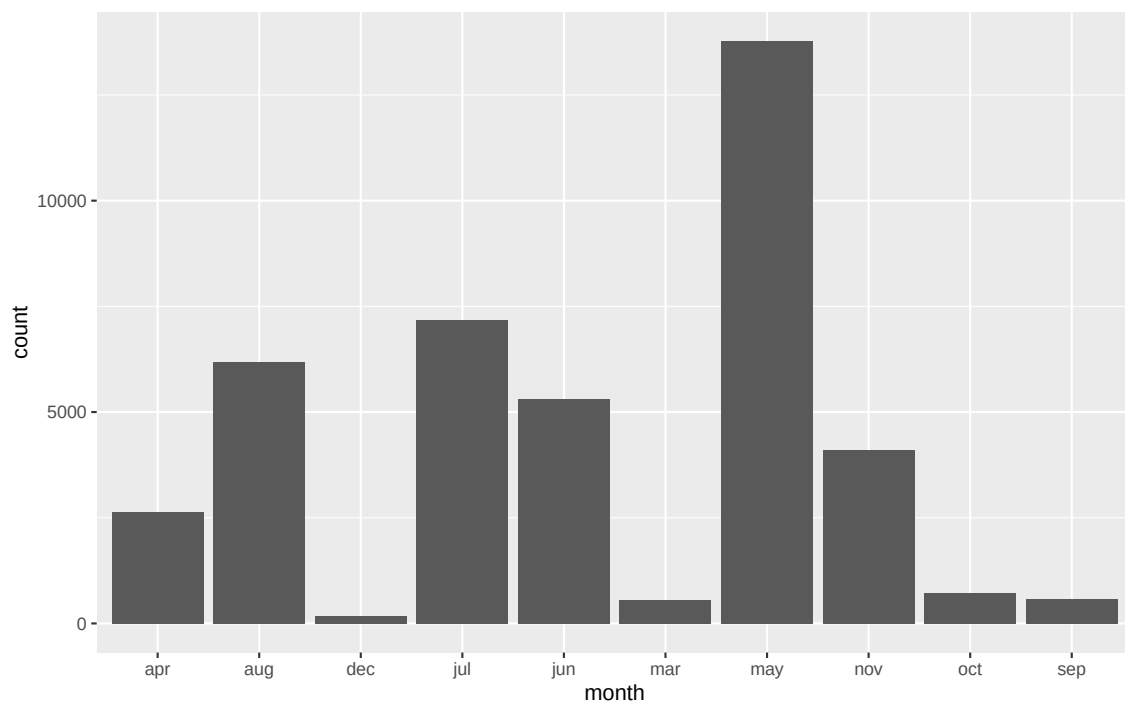
Análisis exploratorio (EDA)

```
# ggplot(datos, aes(y)) inicia el gráfico indicando el dataset y qué
# variable va en el eje x. geom_bar() agrega una capa de barras que cuenta
# automáticamente cuántas observaciones hay por cada valor de "y".
ggplot(datos, aes(y)) +
  geom_bar()
```



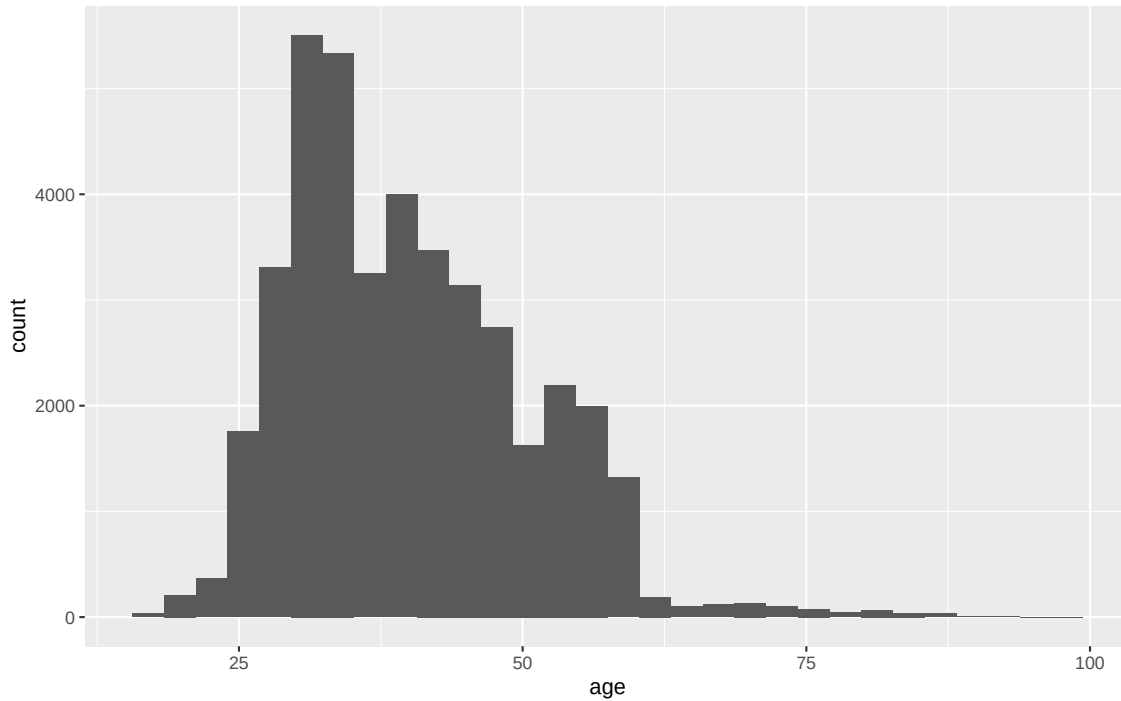
Análisis del resultado. Se confirma el desbalance visto antes: ~89 % "no" vs ~11 % "yes".

```
ggplot(datos, aes(month)) +  
  geom_bar()
```



Análisis del resultado. Distribución intermitente entre meses, posiblemente por campañas concentradas en ciertos periodos.

```
# geom_histogram(bins = 30) agrupa los valores de "age" en 30 intervalos de
# igual ancho y grafica cuántas observaciones caen en cada uno.
ggplot(datos, aes(age)) +
  geom_histogram(bins = 30)
```



Análisis del resultado. Distribución razonable de edades, sin valores extraños.

```
prop.table(table(datos$month, datos$y), 1)
```

```
#>
#>      no    yes
#> apr 0.79513 0.20487
#> aug 0.89394 0.10606
#> dec 0.51099 0.48901
#> jul 0.90961 0.09039
#> jun 0.89489 0.10511
#> mar 0.49451 0.50549
#> may 0.93564 0.06436
#> nov 0.89854 0.10146
#> oct 0.56067 0.43933
#> sep 0.55088 0.44912
```

Análisis del resultado. Los meses más equilibrados son diciembre, marzo, octubre y septiembre; el resto mantiene el desbalance general (~90% "no").

Partición de datos (Train / Test) de forma estratificada

```
# Partición estratificada: mantiene la proporción de clases en train y test.
# Es importante porque el target está desbalanceado (~89%/11%). Con una
# partición aleatoria simple, el test podría quedar con muy pocos "yes".
set.seed(123)
```

```
indices_train <- createDataPartition(datos$y, p = 0.8, list = FALSE)
```

```
train <- datos[indices_train, ]
test  <- datos[-indices_train, ]
```

```
cat("\n=== PREVALENCIA EN TRAIN ===\n")
```

```
#>
#> === PREVALENCIA EN TRAIN ===
```

```
print(prop.table(table(train$y)))
```

```
#>
#>      no      yes
#> 0.8873 0.1127
```

```
cat("\n=== PREVALENCIA EN TEST ===\n")
```

```
#>
#> === PREVALENCIA EN TEST ===
```

```
print(prop.table(table(test$y)))
```

```
#>
#>      no      yes
#> 0.8874 0.1126
```

Análisis del resultado. Ambos conjuntos mantienen la proporción original (~89%/11%). A partir de aquí, test queda congelado hasta la evaluación final de cada modelo.

Preparación de variables para los modelos

Al igual que en el laboratorio, las variables categóricas se transforman en variables indicadoras (dummy) y todas las variables se estandarizan **antes** de entrenar, pero aprendiendo la transformación únicamente con train para evitar Data Leakage hacia test.

```
# dummyVars() aprende, a partir de train, cómo convertir cada variable
# categórica en columnas 0/1 (una por categoría, salvo la de referencia
# gracias a fullRank = TRUE, que evita colinealidad perfecta).
dummy_banco <- dummyVars(
  y ~ .,
  data = train,
  fullRank = TRUE
```

```

)
# Este desajuste ocurre porque alguna categoría poco frecuente (por ejemplo,
# "default = yes", que tiene muy pocos casos en todo el dataset) puede caer
# completamente en train o completamente en test al particionar 80/20,
# generando una columna dummy de más en uno de los dos conjuntos.
# Aplicamos esa misma transformación a train y a test:
x_train <- predict(dummy_banco, newdata = train) %>% as.data.frame()
x_test  <- predict(dummy_banco, newdata = test)  %>% as.data.frame()
# Asegurar que x_test tenga exactamente las mismas columnas que x_train,
# en el mismo orden.

# 1) Identificar columnas faltantes en test (respecto a train)
cols_train <- colnames(x_train)
cols_test  <- colnames(x_test)

faltan <- setdiff(cols_train, cols_test)
sobran <- setdiff(cols_test, cols_train)

# 2) Agregar columnas faltantes con valor 0
if (length(faltan) > 0) {
  for (col in faltan) {
    x_test[[col]] <- 0
  }
}

# 3) Eliminar columnas sobrantes (si las hay)
if (length(sobran) > 0) {
  x_test <- x_test[, !(colnames(x_test) %in% sobran), drop = FALSE]
}

# 4) Reordenar para que coincidan exactamente con train
x_test <- x_test[, cols_train, drop = FALSE]

# Verificación rápida:
cat("¿Coinciden los nombres? ", identical(colnames(x_train), colnames(x_test)), "\n")

```

```
#> ¿Coinciden los nombres? TRUE
```

```

y_train <- train$y
y_test  <- test$y

cat("Cantidad de predictores después de crear variables dummy:\n")

```

```
#> Cantidad de predictores después de crear variables dummy:
```

```
ncol(x_train)
```

```
#> [1] 52
```

Estandarización

```
# Los parámetros de centrado y escalado se calculan SOLO con x_train, y
# luego se aplican también a x_test. Así x_test nunca influye en cómo se
# estandarizan los datos (se evita Data Leakage).
preproc <- preProcess(
  x_train,
  method = c("center", "scale")
)

x_train_std <- predict(preproc, x_train)
x_test_std <- predict(preproc, x_test)

summary(x_train_std[, 1:6])
```

```
#>      age      jobblue-collar  jobentrepreneur  jobhousemaid
#> Min.   :-2.208  Min.   :-0.537  Min.   :-0.191  Min.   :-0.163
#> 1st Qu.: -0.769  1st Qu.: -0.537  1st Qu.: -0.191  1st Qu.: -0.163
#> Median : -0.193  Median : -0.537  Median : -0.191  Median : -0.163
#> Mean   :  0.000  Mean   :  0.000  Mean   :  0.000  Mean   :  0.000
#> 3rd Qu.:  0.671  3rd Qu.: -0.537  3rd Qu.: -0.191  3rd Qu.: -0.163
#> Max.   :  5.565  Max.   :  1.863  Max.   :  5.230  Max.   :  6.144
#> jobmanagement  jobretired
#> Min.   :-0.275  Min.   :-0.209
#> 1st Qu.: -0.275  1st Qu.: -0.209
#> Median : -0.275  Median : -0.209
#> Mean   :  0.000  Mean   :  0.000
#> 3rd Qu.: -0.275  3rd Qu.: -0.209
#> Max.   :  3.634  Max.   :  4.779
```

Análisis del resultado. Las variables transformadas quedan con media cercana a cero y desviación estándar cercana a uno. La misma transformación aprendida en `train` fue aplicada a `test`, evitando Data Leakage — igual que en el laboratorio.

Configuración de validación cruzada

```
# Igual que en el laboratorio: validación cruzada de 5 folds,
# sampling = "down" (string built-in de caret, no una función propia)
# aplica down-sampling automáticamente SOLO dentro de cada fold de
# entrenamiento, dejando test con la prevalencia real. summaryFunction =
# twoClassSummary calcula ROC/Sens/Spec en cada fold, por lo que el
# criterio para elegir hiperparámetros será ROC (AUC), no Accuracy.
control <- trainControl(
  method = "cv",
  number = 5,
  classProbs = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final",
  sampling = "down"
)
```

```
print(control)
```

```
#> $method
#> [1] "cv"
#>
#> $number
#> [1] 5
#>
#> $repeats
#> [1] NA
#>
#> $search
#> [1] "grid"
#>
#> $p
#> [1] 0.75
#>
#> $initialWindow
#> NULL
#>
#> $horizon
#> [1] 1
#>
#> $fixedWindow
#> [1] TRUE
#>
#> $skip
#> [1] 0
#>
#> $verboseIter
#> [1] FALSE
#>
#> $returnData
#> [1] TRUE
#>
#> $returnResamp
#> [1] "final"
#>
#> $savePredictions
#> [1] "final"
#>
#> $classProbs
#> [1] TRUE
#>
#> $summaryFunction
#> function (data, lev = NULL, model = NULL)
#> {
#>   if (length(lev) > 2) {
#>     stop(paste("Your outcome has", length(lev), "levels. The twoClassSummary() function isn't ap
#>   }
#>   requireNamespaceQuietStop("pROC")
#>   if (!all(levels(data[, "pred"]) == lev)) {
#>     stop("levels of observed and predicted data do not match")
```

```

#>   }
#>   rocObject <- try(pROC::roc(data$obs, data[, lev[1]], direction = ">",
#>     quiet = TRUE), silent = TRUE)
#>   rocAUC <- if (inherits(rocObject, "try-error"))
#>     NA
#>   else rocObject$auc
#>   out <- c(rocAUC, sensitivity(data[, "pred"], data[, "obs"],
#>     lev[1]), specificity(data[, "pred"], data[, "obs"], lev[2]))
#>   names(out) <- c("ROC", "Sens", "Spec")
#>   out
#> }
#> <bytecode: 0x1490c2a90>
#> <environment: namespace:caret>
#>
#> $selectionFunction
#> [1] "best"
#>
#> $preProcOptions
#> $preProcOptions$thresh
#> [1] 0.95
#>
#> $preProcOptions$ICAcomp
#> [1] 3
#>
#> $preProcOptions$k
#> [1] 5
#>
#> $preProcOptions$freqCut
#> [1] 19
#>
#> $preProcOptions$uniqueCut
#> [1] 10
#>
#> $preProcOptions$cutoff
#> [1] 0.9
#>
#>
#> $sampling
#> [1] "down"
#>
#> $index
#> NULL
#>
#> $indexOut
#> NULL
#>
#> $indexFinal
#> NULL
#>
#> $timingSamps
#> [1] 0
#>
#> $predictionBounds
#> [1] FALSE FALSE

```

```

#>
#> $seeds
#> [1] NA
#>
#> $adaptive
#> $adaptive$min
#> [1] 5
#>
#> $adaptive$alpha
#> [1] 0.05
#>
#> $adaptive$method
#> [1] "gls"
#>
#> $adaptive$complete
#> [1] TRUE
#>
#>
#> $trim
#> [1] FALSE
#>
#> $allowParallel
#> [1] TRUE

```

Modelo baseline

```

# El baseline es el punto de comparación mínimo exigido por el enunciado:
# un "modelo" que siempre predice la clase mayoritaria de train ("no"),
# sin usar ninguna variable predictora. Cualquier modelo real debe superar
# claramente a este baseline para justificar su uso.
clase_mayoritaria <- names(which.max(table(y_train)))
cat("Clase mayoritaria en train:", clase_mayoritaria, "\n")

```

```

#> Clase mayoritaria en train: no

```

```

pred_baseline <- factor(
  rep(clase_mayoritaria, length(y_test)),
  levels = c("no", "yes")
)

cm_baseline <- confusionMatrix(pred_baseline, y_test, positive = "yes")
print(cm_baseline)

```

```

#> Confusion Matrix and Statistics
#>
#>           Reference
#> Prediction  no  yes
#>          no 7307 927
#>          yes   0   0
#>

```

```

#>             Accuracy : 0.887
#>             95% CI : (0.88, 0.894)
#>      No Information Rate : 0.887
#>      P-Value [Acc > NIR] : 0.509
#>
#>             Kappa : 0
#>
#>  McNemar's Test P-Value : <2e-16
#>
#>      Sensitivity : 0.000
#>      Specificity : 1.000
#>      Pos Pred Value :  NaN
#>      Neg Pred Value : 0.887
#>      Prevalence : 0.113
#>      Detection Rate : 0.000
#>      Detection Prevalence : 0.000
#>      Balanced Accuracy : 0.500
#>
#>      'Positive' Class : yes
#>

```

```

cat("\nAccuracy baseline:", round(cm_baseline$overall["Accuracy"], 4), "\n")

```

```

#>
#> Accuracy baseline: 0.8874

```

```

cat("Sensibilidad baseline:", round(cm_baseline$byClass["Sensitivity"], 4), "\n")

```

```

#> Sensibilidad baseline: 0

```

Análisis del resultado. El baseline logra ~89% de Accuracy solo por predecir siempre “no”, pero con Sensibilidad 0: nunca identifica un cliente que efectivamente acepta el depósito. Confirma que Accuracy sola no sirve como métrica principal en este problema.

=====

REGRESIÓN LOGÍSTICA (CORREGIDA: preProcess dentro de cada fold)

=====

```

# CORRECCIÓN RESPECTO A LA VERSIÓN ANTERIOR: antes calculábamos
# x_train_std/x_test_std con preProcess() UNA SOLA VEZ, afuera de la
# validación cruzada. Eso significa que los parámetros de centrado y
# escalado (media y desviación estándar de cada variable) se calculaban
# viendo TODO train de una vez, incluyendo lo que en cada fold sería la
# porción de validación - una fuga leve de información.

```

```

#
# La corrección es pasar preProcess = c("center", "scale") como
# ARGUMENTO de train(), en vez de aplicarlo manualmente antes. Así,
# caret recalcula la media y desviación estándar DENTRO de cada fold,
# usando solo los datos de entrenamiento de ese fold específico -
# igual de riguroso que como ya hacíamos con el down-sampling.
#
# Por esto también volvemos a usar la fórmula "y ~ ." sobre "train"
# directo, en vez de x_train_std/y_train: train() con fórmula +
# preProcess se encarga de las variables dummy y del escalado
# internamente, fold por fold.
set.seed(123)

glm_fit <- train(
  y ~ .,
  data = train,
  method = "glm",
  family = "binomial",
  metric = "ROC",          # se sigue optimizando por ROC, no Accuracy
  trControl = control,    # mismo control de siempre (CV 5 folds + down-sampling dentro de fold)
  preProcess = c("center", "scale")
)

print(glm_fit)

```

```

#> Generalized Linear Model
#>
#> 32942 samples
#> 19 predictor
#> 2 classes: 'no', 'yes'
#>
#> Pre-processing: centered (52), scaled (52)
#> Resampling: Cross-Validated (5 fold)
#> Summary of sample sizes: 26354, 26354, 26354, 26353, 26353
#> Additional sampling using down-sampling prior to pre-processing
#>
#> Resampling results:
#>
#> ROC      Sens    Spec
#> 0.7947 0.8428 0.6441

```

EVALUACIÓN DE REGRESIÓN LOGÍSTICA EN TEST

```

# Como el modelo ahora se entrenó con "train" original (no con
# x_train_std), evaluamos directo sobre "test" (no x_test_std). caret

```

```
# aplica automáticamente, por dentro, el mismo preProcess aprendido en  
# el entrenamiento final a cualquier "newdata" que le pasemos a predict().
```

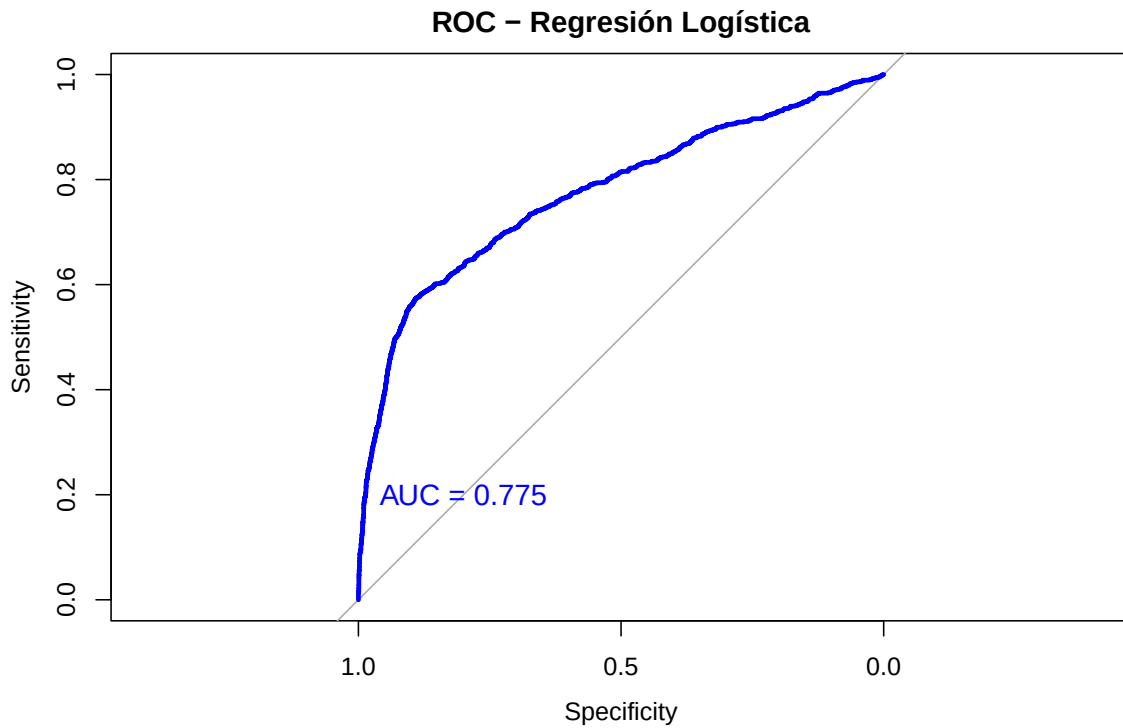
```
pred_logit <- predict(glm_fit, newdata = test)  
  
prob_logit <- predict(glm_fit, newdata = test, type = "prob")[, "yes"]  
  
cm_logit <- confusionMatrix(pred_logit, test$y, positive = "yes")  
print(cm_logit)
```

```
#> Confusion Matrix and Statistics  
#>  
#>           Reference  
#> Prediction  no  yes  
#>         no 6202 369  
#>         yes 1105 558  
#>  
#>           Accuracy : 0.821  
#>           95% CI : (0.813, 0.829)  
#>    No Information Rate : 0.887  
#>    P-Value [Acc > NIR] : 1  
#>  
#>           Kappa : 0.335  
#>  
#> Mcnemar's Test P-Value : <2e-16  
#>  
#>           Sensitivity : 0.6019  
#>           Specificity : 0.8488  
#>    Pos Pred Value : 0.3355  
#>    Neg Pred Value : 0.9438  
#>           Prevalence : 0.1126  
#>    Detection Rate : 0.0678  
#>    Detection Prevalence : 0.2020  
#>    Balanced Accuracy : 0.7254  
#>  
#>    'Positive' Class : yes  
#>
```

```
roc_logit <- roc(test$y, prob_logit, levels = c("no", "yes"), direction = "<")  
auc_logit <- auc(roc_logit)  
cat("AUC Regresión Logística:", round(auc_logit, 4), "\n")
```

```
#> AUC Regresión Logística: 0.7748
```

```
plot(roc_logit, col = "blue", lwd = 3, main = "ROC - Regresión Logística")  
text(0.8, 0.2, paste("AUC =", round(auc_logit, 3)), col = "blue", cex = 1.2)
```



K-NEAREST NEIGHBORS (KNN)

KNN no estima una ecuación como la regresión logística: clasifica una observación nueva según la clase mayoritaria entre sus k vecinos más cercanos, usando distancia euclidiana sobre los predictores ya estandarizados. Por eso usa $x_{train_std}/x_{test_std}$, igual que en el laboratorio, y no la fórmula sobre datos sin escalar.

`expand.grid()` arma la grilla de valores de k a probar. Usamos valores impares para evitar empates en clasificación binaria, igual que la secuencia del laboratorio (`seq(3, 25, by = 2)`).

```
# Mismo motivo que en la regresión logística: KNN depende MUCHO de la
# escala de las variables (calcula distancias), así que es especialmente
# importante que el escalado no tenga fuga de información. Con
# preProcess dentro de train(), cada fold recalcula su propio centrado
# y escalado, en vez de heredar uno calculado sobre todo train.
set.seed(123)
```

```
knn_fit <- train(
  y ~ .,
  data = train,
  method = "knn",
  metric = "ROC",
  trControl = control,
  preProcess = c("center", "scale"),
```

```

tuneGrid = expand.grid(
  k = seq(3, 51, by = 4)  # misma grilla ampliada que ya habíamos definido
)

print(knn_fit)

```

```

#> k-Nearest Neighbors
#>
#> 32942 samples
#> 19 predictor
#> 2 classes: 'no', 'yes'
#>
#> Pre-processing: centered (52), scaled (52)
#> Resampling: Cross-Validated (5 fold)
#> Summary of sample sizes: 26354, 26354, 26354, 26353, 26353
#> Additional sampling using down-sampling prior to pre-processing
#>
#> Resampling results across tuning parameters:
#>
#>  k   ROC     Sens   Spec
#>  3  0.7188  0.6962  0.6595
#>  7  0.7510  0.7492  0.6563
#> 11  0.7690  0.7791  0.6633
#> 15  0.7699  0.7936  0.6501
#> 19  0.7738  0.8036  0.6439
#> 23  0.7777  0.8117  0.6487
#> 27  0.7790  0.8117  0.6447
#> 31  0.7762  0.8187  0.6363
#> 35  0.7802  0.8239  0.6325
#> 39  0.7803  0.8327  0.6263
#> 43  0.7823  0.8320  0.6290
#> 47  0.7835  0.8337  0.6307
#> 51  0.7822  0.8391  0.6220
#>
#> ROC was used to select the optimal model using the largest value.
#> The final value used for the model was k = 47.

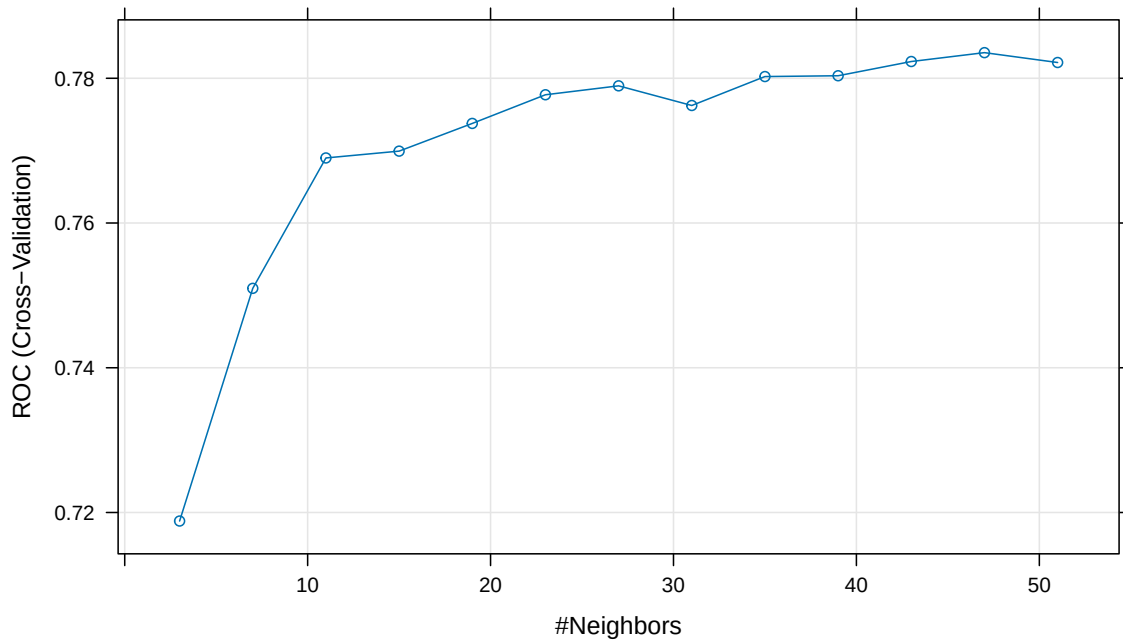
```

```

plot(knn_fit, main = "ROC de validación cruzada según k (KNN)")

```

ROC de validación cruzada según k (KNN)



EVALUACIÓN DE KNN EN TEST

```
# Mismo motivo que en la regresión logística: KNN depende MUCHO de la  
# escala de las variables (calcula distancias), así que es especialmente  
# importante que el escalado no tenga fuga de información. Con  
# preProcess dentro de train(), cada fold recalcula su propio centrado  
# y escalado, en vez de heredar uno calculado sobre todo train.  
set.seed(123)  
  
knn_fit <- train(  
  y ~ .,  
  data = train,  
  method = "knn",  
  metric = "ROC",  
  trControl = control,  
  preProcess = c("center", "scale"),  
  tuneGrid = expand.grid(  
    k = seq(3, 51, by = 4) # misma grilla ampliada que ya habíamos definido  
  )  
)  
  
print(knn_fit)
```

```

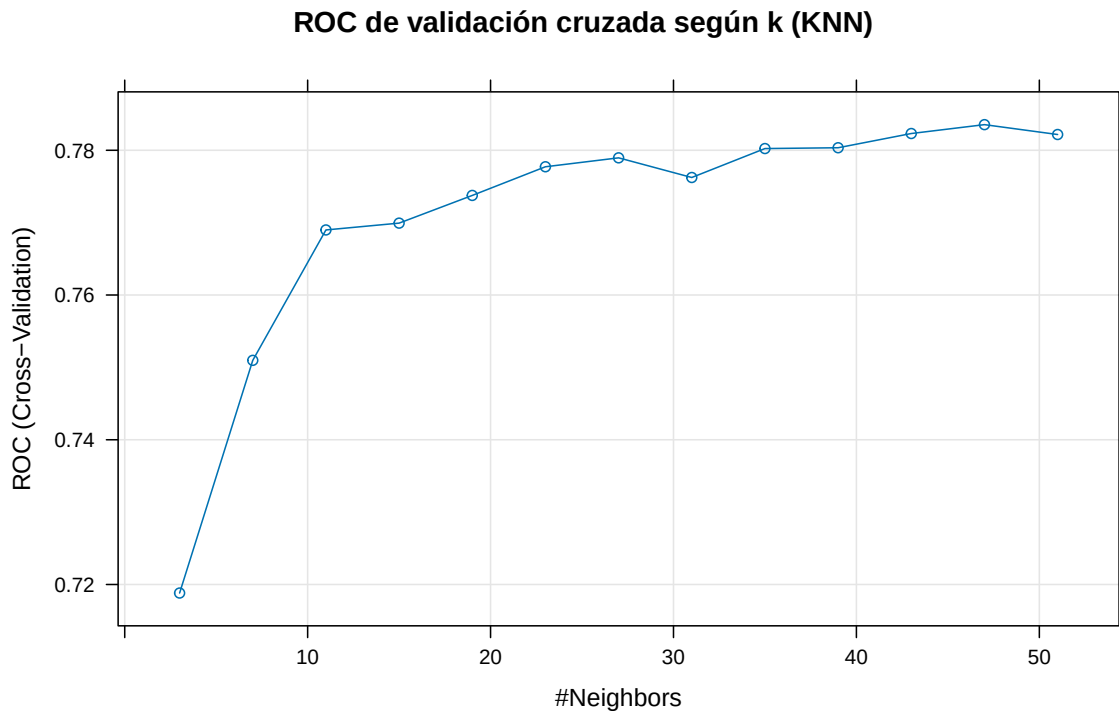
#> k-Nearest Neighbors
#>
#> 32942 samples
#> 19 predictor
#> 2 classes: 'no', 'yes'
#>
#> Pre-processing: centered (52), scaled (52)
#> Resampling: Cross-Validated (5 fold)
#> Summary of sample sizes: 26354, 26354, 26354, 26353, 26353
#> Additional sampling using down-sampling prior to pre-processing
#>
#> Resampling results across tuning parameters:
#>
#>   k   ROC     Sens   Spec
#>   3 0.7188 0.6962 0.6595
#>   7 0.7510 0.7492 0.6563
#>  11 0.7690 0.7791 0.6633
#>  15 0.7699 0.7936 0.6501
#>  19 0.7738 0.8036 0.6439
#>  23 0.7777 0.8117 0.6487
#>  27 0.7790 0.8117 0.6447
#>  31 0.7762 0.8187 0.6363
#>  35 0.7802 0.8239 0.6325
#>  39 0.7803 0.8327 0.6263
#>  43 0.7823 0.8320 0.6290
#>  47 0.7835 0.8337 0.6307
#>  51 0.7822 0.8391 0.6220
#>
#> ROC was used to select the optimal model using the largest value.
#> The final value used for the model was k = 47.

```

```

plot(knn_fit, main = "ROC de validación cruzada según k (KNN)")

```



SVM CON KERNEL LINEAL (CON PROCESAMIENTO EN PARALELO)

SVM es uno de los modelos más lentos de entrenar en datasets grandes, porque su optimización interna (encontrar el hiperplano de margen máximo) escala mal con la cantidad de observaciones. Con ~26,000 filas por fold de entrenamiento y 4 valores de C a probar (4 combinaciones x 5 folds = 20 ajustes de SVM), el tiempo de cómputo puede ser alto si se corre todo de forma secuencial (un ajuste a la vez).

doParallel permite que caret reparta esos 20 ajustes entre varios núcleos del procesador, entrenándolos simultáneamente en vez de uno por uno. No cambia el resultado (siguen siendo los mismos folds, la misma grilla, el mismo down-sampling dentro de cada fold): solo aprovecha mejor el hardware disponible para que termine más rápido.

```
#install.packages("doParallel") # descomentar si no lo tienes instalado
library(doParallel)
```

detectCores() cuenta los núcleos físicos/lógicos del procesador. Se usa "- 1" para dejar un núcleo libre, de modo que el resto del computador (RStudio, Windows, otras aplicaciones) no se quede sin recursos mientras el modelo entrena.

```
nucleos_disponibles <- detectCores() - 1
cat("Núcleos que se usarán en paralelo:", nucleos_disponibles, "\n")
```

```
#> Núcleos que se usarán en paralelo: 7
```

makeCluster() crea el grupo de procesos paralelos, y registerDoParallel() le dice a caret (específicamente a train(), a través de foreach) que use ese cluster para repartir el trabajo. A partir de aquí, cualquier train() que se llame automáticamente corre en paralelo, porque trainControl() ya tiene allowParallel = TRUE por defecto.

```
cl <- makeCluster(nucleos_disponibles)
registerDoParallel(cl)
```

SVM busca el hiperplano que separa las clases maximizando el margen entre ellas. El único hiperparámetro del kernel lineal es C, el costo de penalización por observaciones mal clasificadas o dentro del margen: C chico -> margen más ancho, más tolerante a errores (menor varianza). C grande -> margen más estrecho, ajusta más al train (mayor varianza).

```
set.seed(123)

svm_lineal <- train(
  y ~ .,
  data = train,
  method = "svmLinear",
  metric = "ROC",
  trControl = control,
  preProcess = c("center", "scale"),
  tuneGrid = expand.grid(
    C = c(0.01, 0.1, 1, 10)
  )
)

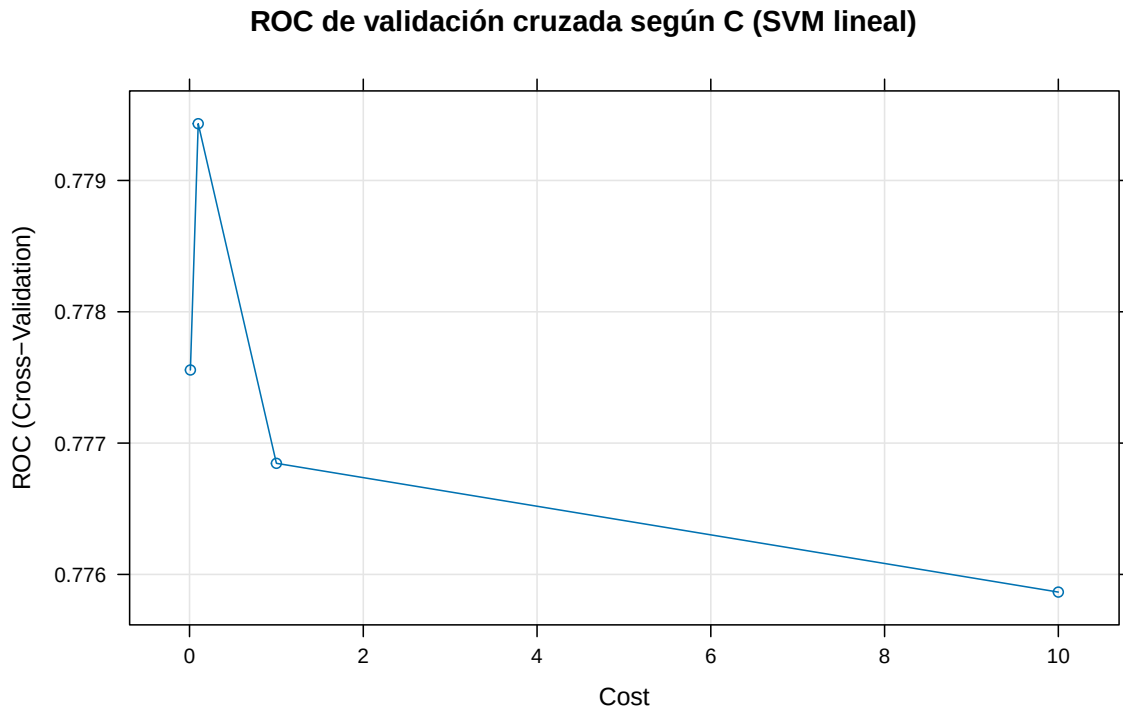
# Cerramos el cluster paralelo apenas termina el entrenamiento, antes de
# revisar resultados - libera los núcleos reservados para que el resto
# del computador vuelva a la normalidad.
stopCluster(cl)
registerDoSEQ()

print(svm_lineal)
```

```
#> Support Vector Machines with Linear Kernel
#>
#> 32942 samples
#> 19 predictor
#> 2 classes: 'no', 'yes'
#>
#> Pre-processing: centered (52), scaled (52)
#> Resampling: Cross-Validated (5 fold)
#> Summary of sample sizes: 26354, 26354, 26354, 26353, 26353
#> Additional sampling using down-sampling prior to pre-processing
#>
#> Resampling results across tuning parameters:
#>
#>  C      ROC      Sens    Spec
#>  0.01  0.7776  0.7299  0.7155
#>  0.10  0.7794  0.8606  0.6218
```

```
#>    1.00  0.7768  0.8616  0.6199
#>   10.00  0.7759  0.8610  0.6215
#>
#> ROC was used to select the optimal model using the largest value.
#> The final value used for the model was C = 0.1.
```

```
plot(svm_lineal, main = "ROC de validación cruzada según C (SVM lineal)")
```



EVALUACIÓN DE SVM LINEAL EN TEST

```
pred_svm_lineal <- predict(svm_lineal, newdata = test)

prob_svm_lineal <- predict(svm_lineal, newdata = test, type = "prob")[, "yes"]

cm_svm_lineal <- confusionMatrix(pred_svm_lineal, test$y, positive = "yes")
print(cm_svm_lineal)
```

```
#> Confusion Matrix and Statistics
#>
#>           Reference
#> Prediction  no  yes
```

```

#>      no  6312  375
#>      yes  995  552
#>
#>              Accuracy : 0.834
#>              95% CI : (0.825, 0.842)
#>      No Information Rate : 0.887
#>      P-Value [Acc > NIR] : 1
#>
#>              Kappa : 0.355
#>
#>      McNemar's Test P-Value : <2e-16
#>
#>      Sensitivity : 0.595
#>      Specificity : 0.864
#>      Pos Pred Value : 0.357
#>      Neg Pred Value : 0.944
#>      Prevalence : 0.113
#>      Detection Rate : 0.067
#>      Detection Prevalence : 0.188
#>      Balanced Accuracy : 0.730
#>
#>      'Positive' Class : yes
#>

```

```

roc_svm_lineal <- roc(test$y, prob_svm_lineal, levels = c("no", "yes"), direction = "<")
auc_svm_lineal <- auc(roc_svm_lineal)
cat("AUC SVM lineal:", round(auc_svm_lineal, 4), "\n")

```

```

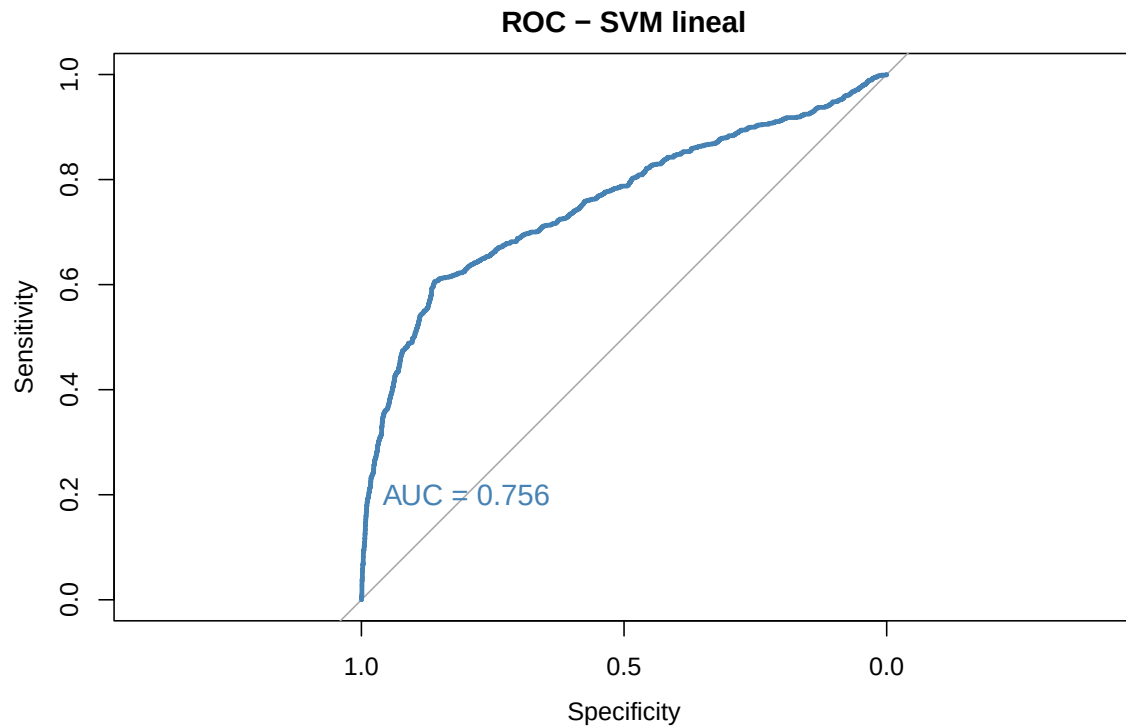
#> AUC SVM lineal: 0.7562

```

```

plot(roc_svm_lineal, col = "steelblue", lwd = 3, main = "ROC - SVM lineal")
text(0.8, 0.2, paste("AUC =", round(auc_svm_lineal, 3)), col = "steelblue", cex = 1.2)

```



SVM CON KERNEL RBF (CON PROCESAMIENTO EN PARALELO)

```
# Mismo criterio que SVM lineal, pero con el kernel radial (sigma + C).  
# Usamos la grilla original (no la ampliada) porque el sigma/C que  
# habíamos explorado no tenía margen claro para ampliar sin gastar mucho  
# más tiempo de cómputo, dado que ya vamos justos de días.  
library(doParallel)  
nucleos_disponibles <- detectCores() - 1  
cl <- makeCluster(nucleos_disponibles)  
registerDoParallel(cl)  
  
set.seed(123)  
  
svm_rbf <- train(  
  y ~ .,  
  data = train,  
  method = "svmRadial",  
  metric = "ROC",  
  trControl = control,  
  preProcess = c("center", "scale"),
```

```

tuneGrid = expand.grid(
  sigma = c(0.01, 0.05),
  C = c(0.5, 1)
)

stopCluster(cl)
registerDoSEQ()

print(svm_rbf)

```

```

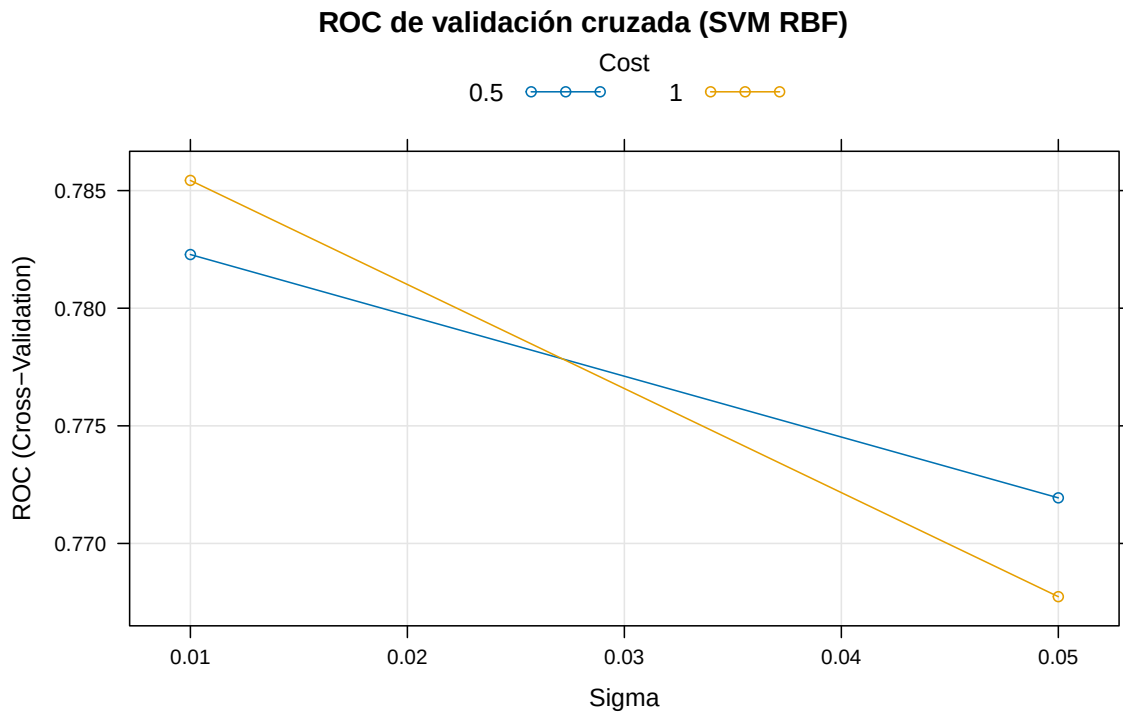
#> Support Vector Machines with Radial Basis Function Kernel
#>
#> 32942 samples
#> 19 predictor
#> 2 classes: 'no', 'yes'
#>
#> Pre-processing: centered (52), scaled (52)
#> Resampling: Cross-Validated (5 fold)
#> Summary of sample sizes: 26354, 26354, 26354, 26353, 26353
#> Additional sampling using down-sampling prior to pre-processing
#>
#> Resampling results across tuning parameters:
#>
#>  sigma  C    ROC      Sens   Spec
#>  0.01   0.5  0.7823  0.8096  0.6754
#>  0.01   1.0  0.7854  0.8242  0.6639
#>  0.05   0.5  0.7719  0.7917  0.6800
#>  0.05   1.0  0.7677  0.7858  0.6781
#>
#> ROC was used to select the optimal model using the largest value.
#> The final values used for the model were sigma = 0.01 and C = 1.

```

```

plot(svm_rbf, main = "ROC de validación cruzada (SVM RBF)")

```



EVALUACIÓN DE SVM RBF EN TEST

```
pred_svm_rbf <- predict(svm_rbf, newdata = test)

prob_svm_rbf <- predict(svm_rbf, newdata = test, type = "prob")[, "yes"]

cm_svm_rbf <- confusionMatrix(pred_svm_rbf, test$y, positive = "yes")
print(cm_svm_rbf)
```

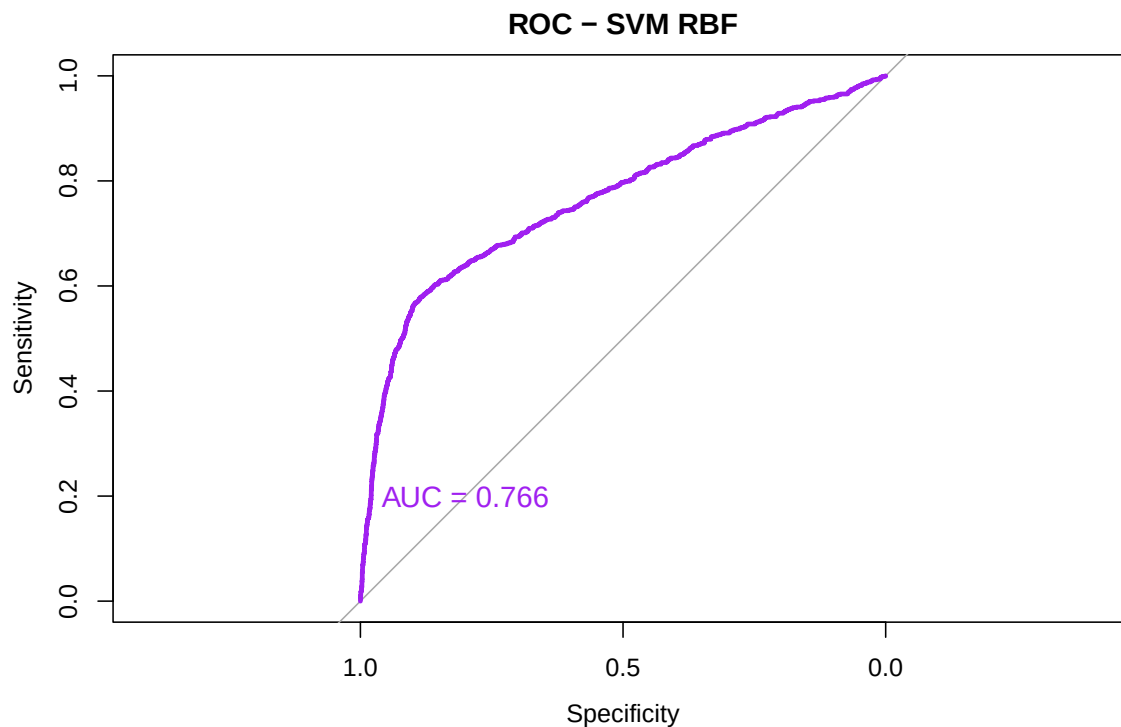
```
#> Confusion Matrix and Statistics
#>
#>      Reference
#> Prediction  no  yes
#>      no  6250  368
#>      yes 1057  559
#>
#>      Accuracy : 0.827
#>      95% CI   : (0.819, 0.835)
#> No Information Rate : 0.887
#> P-Value [Acc > NIR] : 1
#>
#>      Kappa   : 0.346
```

```
#>
#> McNemar's Test P-Value : <2e-16
#>
#>      Sensitivity : 0.6030
#>      Specificity : 0.8553
#>      Pos Pred Value : 0.3459
#>      Neg Pred Value : 0.9444
#>      Prevalence : 0.1126
#>      Detection Rate : 0.0679
#>      Detection Prevalence : 0.1963
#>      Balanced Accuracy : 0.7292
#>
#>      'Positive' Class : yes
#>
```

```
roc_svm_rbf <- roc(test$y, prob_svm_rbf, levels = c("no", "yes"), direction = "<")
auc_svm_rbf <- auc(roc_svm_rbf)
cat("AUC SVM RBF:", round(auc_svm_rbf, 4), "\n")
```

```
#> AUC SVM RBF: 0.7659
```

```
plot(roc_svm_rbf, col = "purple", lwd = 3, main = "ROC - SVM RBF")
text(0.8, 0.2, paste("AUC =", round(auc_svm_rbf, 3)), col = "purple", cex = 1.2)
```



ÁRBOL DE CLASIFICACIÓN

```
# A diferencia de KNN y SVM, los árboles (rpart) NO dependen de la escala  
# de las variables ni necesitan variables dummy: las divisiones se hacen  
# variable por variable comparando contra un umbral, y rpart maneja  
# directamente las variables categóricas como factores. Por eso usamos  
# train (el original, sin estandarizar/con dummies), no x_train_std.  
#  
# El hiperparámetro cp (complexity parameter) controla qué tan permisivo  
# es el árbol para seguir creciendo: cp chico -> árbol más grande y  
# complejo (riesgo de sobreajuste); cp grande -> árbol más podado (riesgo  
# de subajuste).
```

```
set.seed(123)
```

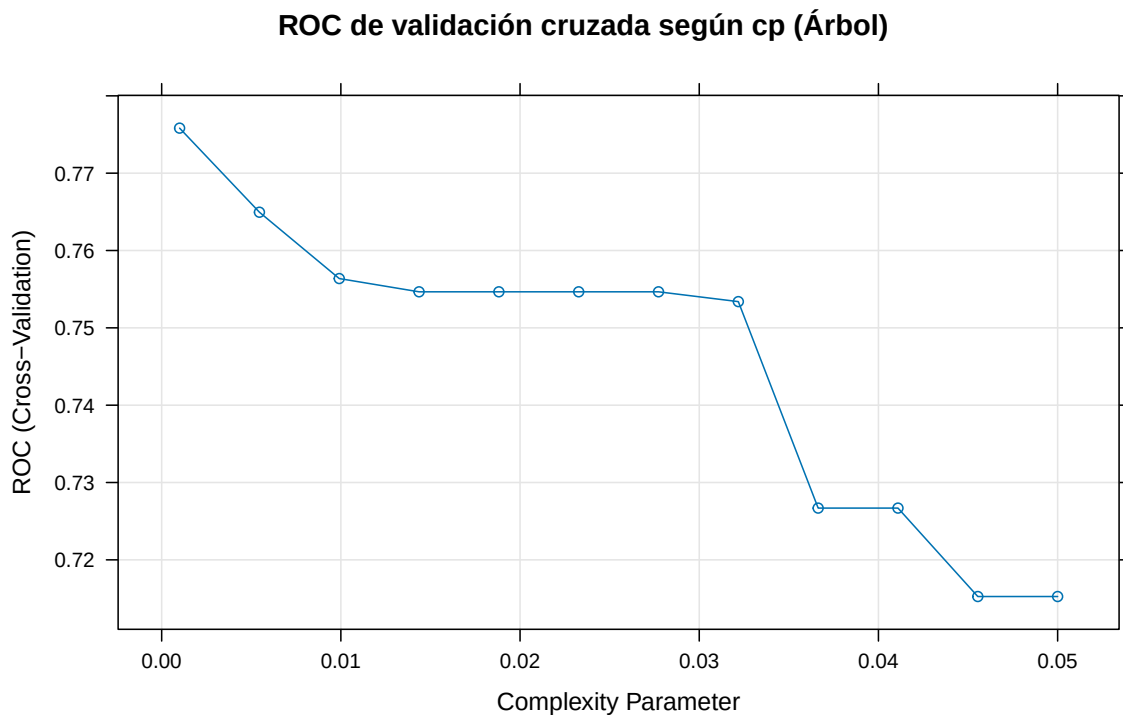
```
arbol_fit <- train(  
  y ~ .,  
  data = train,  
  method = "rpart",  
  metric = "ROC",  
  trControl = control,  
  tuneGrid = expand.grid(  
    cp = seq(0.001, 0.05, length.out = 12)  
  )  
)
```

```
print(arbol_fit)
```

```
#> CART  
#>  
#> 32942 samples  
#>   19 predictor  
#>   2 classes: 'no', 'yes'  
#>  
#> No pre-processing  
#> Resampling: Cross-Validated (5 fold)  
#> Summary of sample sizes: 26354, 26354, 26354, 26353, 26353  
#> Additional sampling using down-sampling  
#>  
#> Resampling results across tuning parameters:  
#>  
#>   cp      ROC    Sens   Spec  
#> 0.001000 0.7758 0.8512 0.6485  
#> 0.005455 0.7650 0.8431 0.6377  
#> 0.009909 0.7564 0.8618 0.6110  
#> 0.014364 0.7547 0.8620 0.6080  
#> 0.018818 0.7547 0.8620 0.6080
```

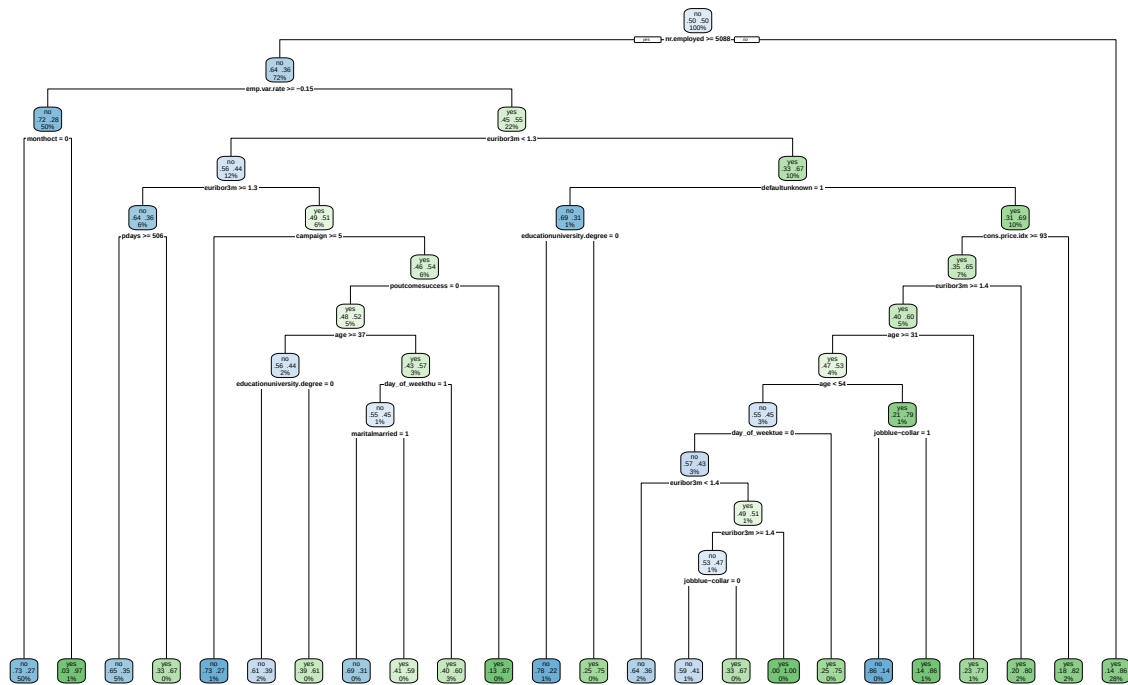
```
#> 0.023273 0.7547 0.8620 0.6080
#> 0.027727 0.7547 0.8620 0.6080
#> 0.032182 0.7534 0.8338 0.6282
#> 0.036636 0.7267 0.8591 0.5698
#> 0.041091 0.7267 0.8591 0.5698
#> 0.045545 0.7153 0.9001 0.5224
#> 0.050000 0.7153 0.9001 0.5224
#>
#> ROC was used to select the optimal model using the largest value.
#> The final value used for the model was cp = 0.001.
```

```
plot(arbol_fit, main = "ROC de validación cruzada según cp (Árbol)")
```



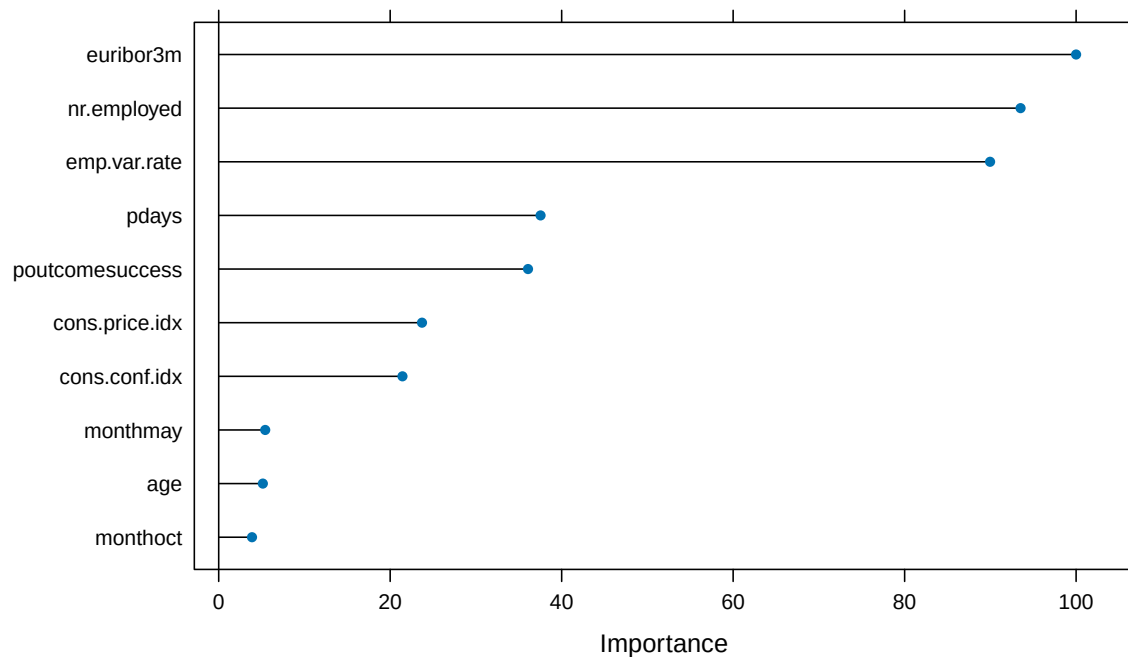
VISUALIZACIÓN E IMPORTANCIA DE VARIABLES

```
rpart.plot(
  arbol_fit$finalModel,
  type = 2,
  extra = 104,
  fallen.leaves = TRUE,
  tweak = 0.8
)
```



```
varImp(arbol_fit) %>%
  plot(top = 10, main = "Importancia de variables en el árbol")
```

Importancia de variables en el árbol



EVALUACIÓN DEL ÁRBOL EN TEST

```
pred_arbol <- predict(arbol_fit, newdata = test)

prob_arbol <- predict(
  arbol_fit,
  newdata = test,
  type = "prob"
)[, "yes"]

cm_arbol <- confusionMatrix(pred_arbol, test$y, positive = "yes")
print(cm_arbol)
```

```
#> Confusion Matrix and Statistics
#>
#>           Reference
#> Prediction  no  yes
#>          no 6274 362
#>          yes 1033 565
#>
#>           Accuracy : 0.831
#>           95% CI : (0.822, 0.839)
#>    No Information Rate : 0.887
#>    P-Value [Acc > NIR] : 1
#>
#>           Kappa : 0.356
#>
#>  McNemar's Test P-Value : <2e-16
#>
#>           Sensitivity : 0.6095
#>           Specificity : 0.8586
#>           Pos Pred Value : 0.3536
#>           Neg Pred Value : 0.9454
#>           Prevalence : 0.1126
#>           Detection Rate : 0.0686
#>    Detection Prevalence : 0.1941
#>           Balanced Accuracy : 0.7341
#>
#>           'Positive' Class : yes
#>
```

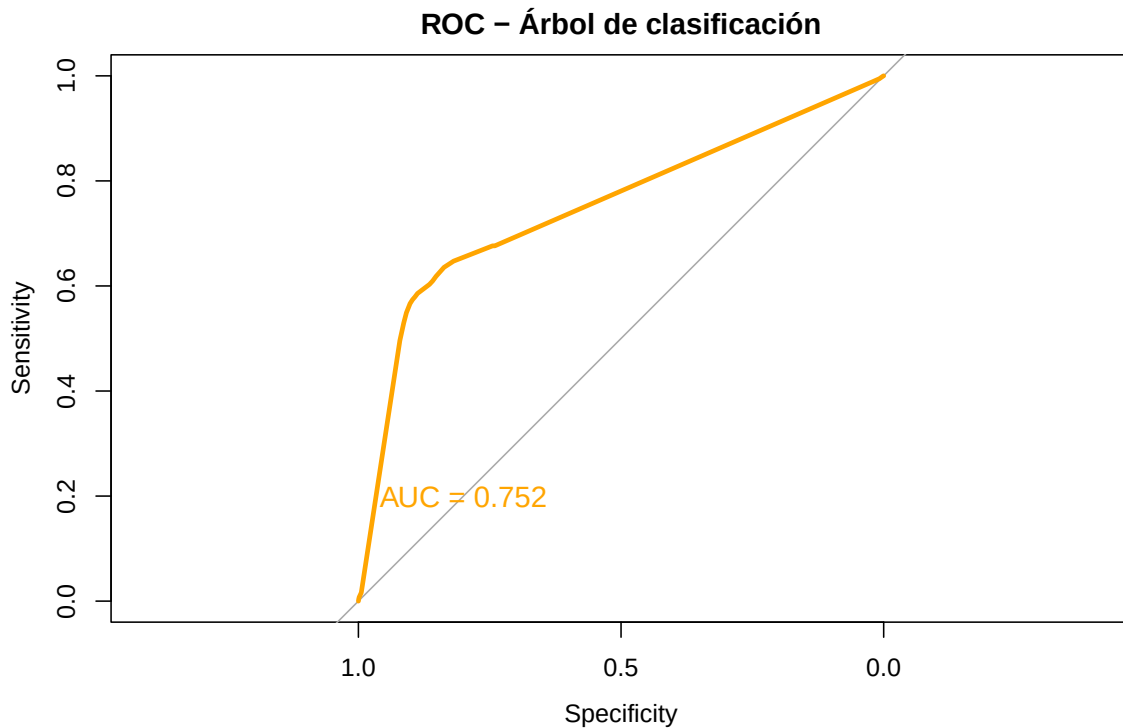
```
roc_arbol <- roc(
  test$y,
  prob_arbol,
  levels = c("no", "yes"),
  direction = "<"
```

```
)
auc_arbol <- auc(roc_arbol)

cat("AUC Árbol:", round(auc_arbol, 4), "\n")
```

```
#> AUC Árbol: 0.7516
```

```
plot(roc_arbol, col = "orange", lwd = 3, main = "ROC - Árbol de clasificación")
text(0.8, 0.2, paste("AUC =", round(auc_arbol, 3)), col = "orange", cex = 1.2)
```



RANDOM FOREST

```
# Random Forest es un ensamble de muchos árboles entrenados sobre muestras
# bootstrap de los datos, donde además cada división del árbol solo puede
# usar un subconjunto aleatorio de "mtry" variables. Esto decorrelaciona
# los árboles entre sí y reduce la varianza respecto a un único árbol
# (como vimos en la tabla comparativa del laboratorio, RF suele superar
# al árbol individual).
#
# Igual que el árbol, Random Forest no depende de la escala de las
```

```

# variables, así que entrena sobre "train" original, no x_train_std.
#
# El hiperparámetro que caret permite tunear vía train() es "mtry"
# (cantidad de variables candidatas en cada división). La cantidad de
# árboles (ntree) se fija aparte, no forma parte de la grilla de caret.

library(doParallel)
nucleos_disponibles <- detectCores() - 1
cl <- makeCluster(nucleos_disponibles)
registerDoParallel(cl)

set.seed(123)

n_predictores <- ncol(train) - 1 # todas las columnas menos "y"

rf_fit <- train(
  y ~ .,
  data = train,
  method = "rf",
  metric = "ROC",
  trControl = control,
  tuneGrid = expand.grid(
    mtry = c(2, floor(sqrt(n_predictores)), floor(n_predictores / 2))
  ),
  ntree = 300 # cantidad de árboles del bosque, fijada fuera de la grilla
)

stopCluster(cl)
registerDoSEQ()

print(rf_fit)

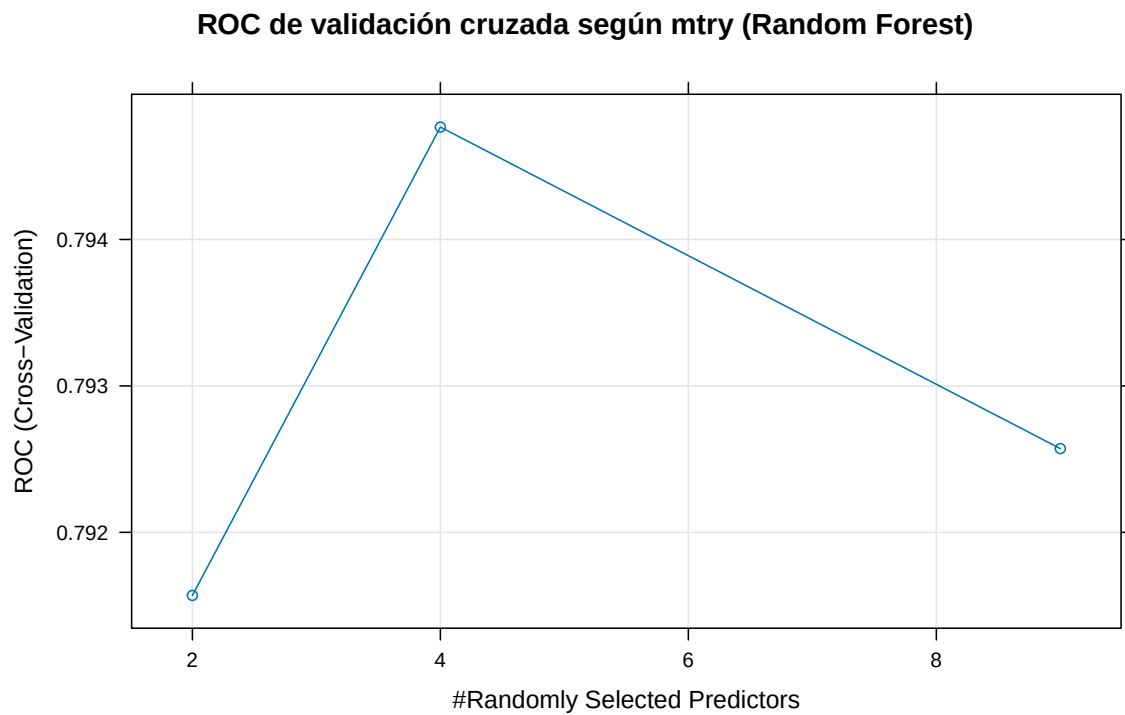
```

```

#> Random Forest
#>
#> 32942 samples
#> 19 predictor
#> 2 classes: 'no', 'yes'
#>
#> No pre-processing
#> Resampling: Cross-Validated (5 fold)
#> Summary of sample sizes: 26354, 26354, 26354, 26353, 26353
#> Additional sampling using down-sampling
#>
#> Resampling results across tuning parameters:
#>
#>   mtry  ROC      Sens   Spec
#>  2    0.7916  0.8493  0.6444
#>  4    0.7948  0.8607  0.6398
#>  9    0.7926  0.8193  0.6670
#>
#> ROC was used to select the optimal model using the largest value.
#> The final value used for the model was mtry = 4.

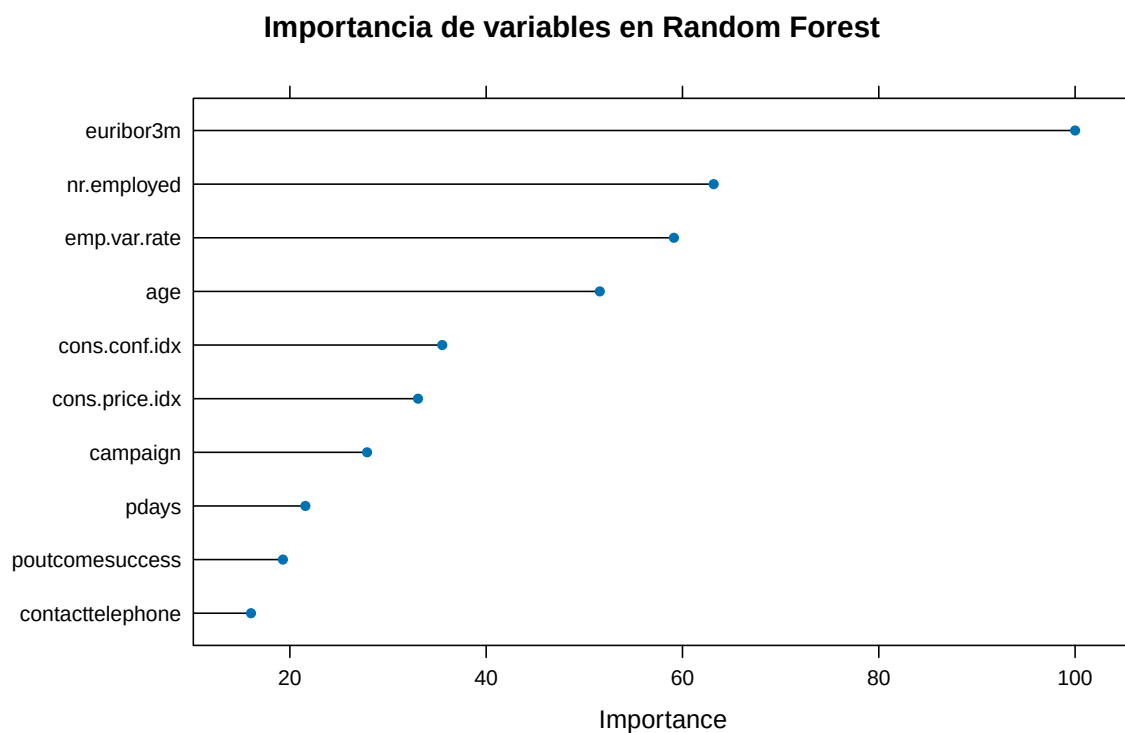
```

```
plot(rf_fit, main = "ROC de validación cruzada según mtry (Random Forest)")
```



#importancia variables

```
varImp(rf_fit) %>%  
  plot(top = 10, main = "Importancia de variables en Random Forest")
```



EVALUACIÓN DE RANDOM FOREST EN TEST

```
pred_rf <- predict(rf_fit, newdata = test)

prob_rf <- predict(
  rf_fit,
  newdata = test,
  type = "prob"
)[, "yes"]

cm_rf <- confusionMatrix(pred_rf, test$y, positive = "yes")
print(cm_rf)
```

```
#> Confusion Matrix and Statistics
#>
#>           Reference
#> Prediction  no  yes
#>          no 6358 368
#>          yes  949 559
#>
#>              Accuracy : 0.84
#>              95% CI : (0.832, 0.848)
#>    No Information Rate : 0.887
#>    P-Value [Acc > NIR] : 1
#>
#>              Kappa : 0.371
#>
#>  McNemar's Test P-Value : <2e-16
#>
#>      Sensitivity : 0.6030
#>      Specificity : 0.8701
#>    Pos Pred Value : 0.3707
#>    Neg Pred Value : 0.9453
#>      Prevalence : 0.1126
#>    Detection Rate : 0.0679
#>  Detection Prevalence : 0.1831
#>    Balanced Accuracy : 0.7366
#>
#>    'Positive' Class : yes
#>
```

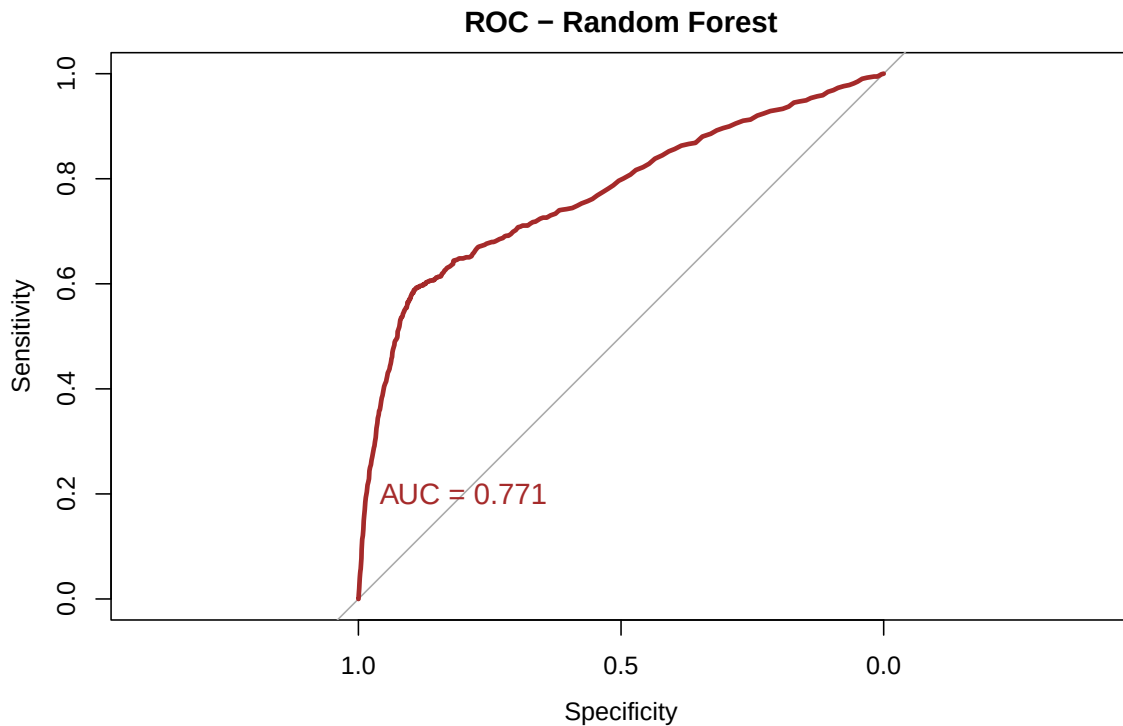
```
roc_rf <- roc(
  test$y,
  prob_rf,
  levels = c("no", "yes"),
  direction = "<"
```

```
)
auc_rf <- auc(roc_rf)

cat("AUC Random Forest:", round(auc_rf, 4), "\n")
```

```
#> AUC Random Forest: 0.7712
```

```
plot(roc_rf, col = "brown", lwd = 3, main = "ROC - Random Forest")
text(0.8, 0.2, paste("AUC =", round(auc_rf, 3)), col = "brown", cex = 1.2)
```



GRADIENT BOOSTING

```
# Gradient Boosting también es un ensamble de árboles, pero a diferencia
# de Random Forest (árboles en paralelo, para reducir varianza), los
# árboles de boosting se construyen de forma SECUENCIAL: cada nuevo árbol
# intenta corregir los errores (residuos) de los árboles anteriores, para
# reducir sesgo. Tiene más hiperparámetros que ajustar:
#   - n.trees: cantidad de árboles/iteraciones de boosting
#   - interaction.depth: profundidad máxima de cada árbol (complejidad)
#   - shrinkage: tasa de aprendizaje (qué tanto corrige cada árbol nuevo)
```

```

# - n.minobsinnode: mínimo de observaciones por nodo terminal
#
# Igual que árbol y Random Forest, no depende de la escala de las
# variables, así que entrena sobre "train" original.

library(doParallel)
nucleos_disponibles <- detectCores() - 1
cl <- makeCluster(nucleos_disponibles)
registerDoParallel(cl)

set.seed(123)

gbm_fit <- train(
  y ~ .,
  data = train,
  method = "gbm",
  metric = "ROC",
  trControl = control,
  tuneGrid = expand.grid(
    n.trees = c(100, 200),
    interaction.depth = c(1, 3),
    shrinkage = c(0.01, 0.1),
    n.minobsinnode = 10
  ),
  verbose = FALSE # evita que gbm imprima el detalle de cada iteración
)

stopCluster(cl)
registerDoSEQ()

print(gbm_fit)

```

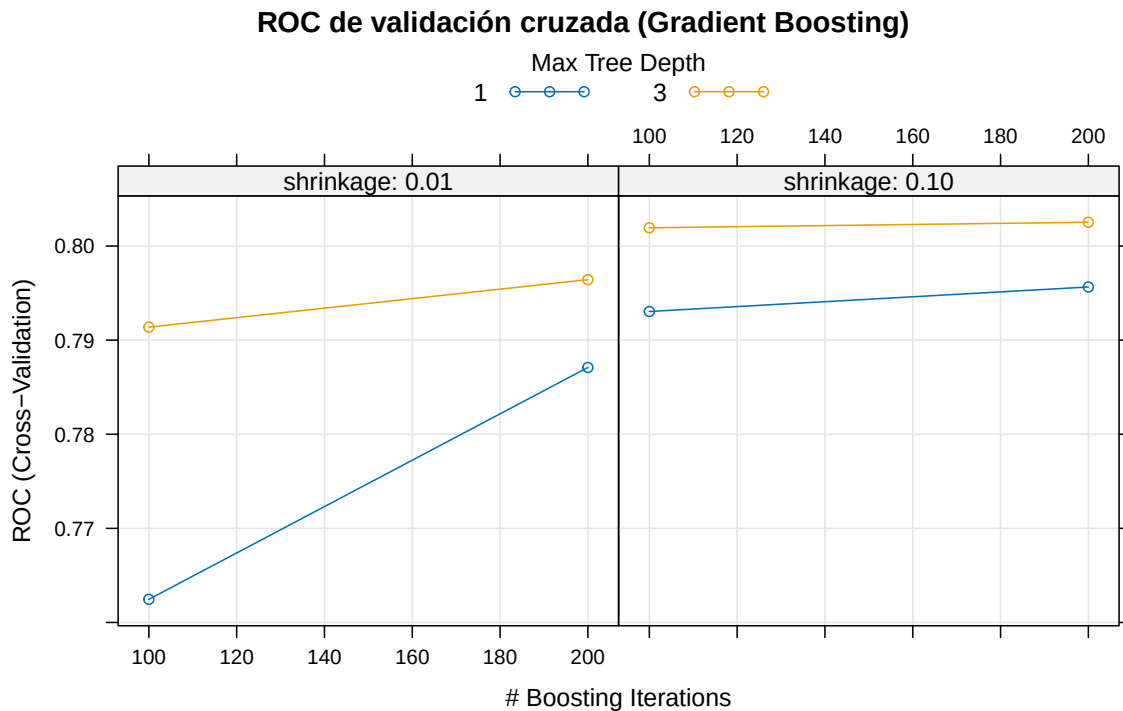
```

#> Stochastic Gradient Boosting
#>
#> 32942 samples
#> 19 predictor
#> 2 classes: 'no', 'yes'
#>
#> No pre-processing
#> Resampling: Cross-Validated (5 fold)
#> Summary of sample sizes: 26354, 26354, 26354, 26353, 26353
#> Additional sampling using down-sampling
#>
#> Resampling results across tuning parameters:
#>
#> shrinkage interaction.depth n.trees ROC Sens Spec
#> 0.01 1 100 0.7625 0.7210 0.7185
#> 0.01 1 200 0.7871 0.7210 0.7185
#> 0.01 3 100 0.7914 0.8506 0.6363
#> 0.01 3 200 0.7964 0.8326 0.6573
#> 0.10 1 100 0.7930 0.8482 0.6277
#> 0.10 1 200 0.7956 0.8476 0.6352
#> 0.10 3 100 0.8019 0.8450 0.6554

```

```
#> 0.10      3      200      0.8025 0.8440 0.6576
#>
#> Tuning parameter 'n.minobsinnode' was held constant at a value of 10
#> ROC was used to select the optimal model using the largest value.
#> The final values used for the model were n.trees = 200, interaction.depth =
#> 3, shrinkage = 0.1 and n.minobsinnode = 10.
```

```
plot(gbm_fit, main = "ROC de validación cruzada (Gradient Boosting)")
```



EVALUACIÓN DE GRADIENT BOOSTING EN TEST

```
pred_gbm <- predict(gbm_fit, newdata = test)

prob_gbm <- predict(
  gbm_fit,
  newdata = test,
  type = "prob"
)[, "yes"]

cm_gbm <- confusionMatrix(pred_gbm, test$y, positive = "yes")
print(cm_gbm)
```

```

#> Confusion Matrix and Statistics
#>
#>           Reference
#> Prediction  no  yes
#>         no 6259 355
#>         yes 1048 572
#>
#>           Accuracy : 0.83
#>           95% CI : (0.821, 0.838)
#>    No Information Rate : 0.887
#>    P-Value [Acc > NIR] : 1
#>
#>           Kappa : 0.357
#>
#> Mcnemar's Test P-Value : <2e-16
#>
#>           Sensitivity : 0.6170
#>           Specificity : 0.8566
#>           Pos Pred Value : 0.3531
#>           Neg Pred Value : 0.9463
#>           Prevalence : 0.1126
#>           Detection Rate : 0.0695
#>    Detection Prevalence : 0.1967
#>           Balanced Accuracy : 0.7368
#>
#>           'Positive' Class : yes
#>

```

```

roc_gbm <- roc(
  test$y,
  prob_gbm,
  levels = c("no", "yes"),
  direction = "<"
)
auc_gbm <- auc(roc_gbm)

cat("AUC Gradient Boosting:", round(auc_gbm, 4), "\n")

```

```

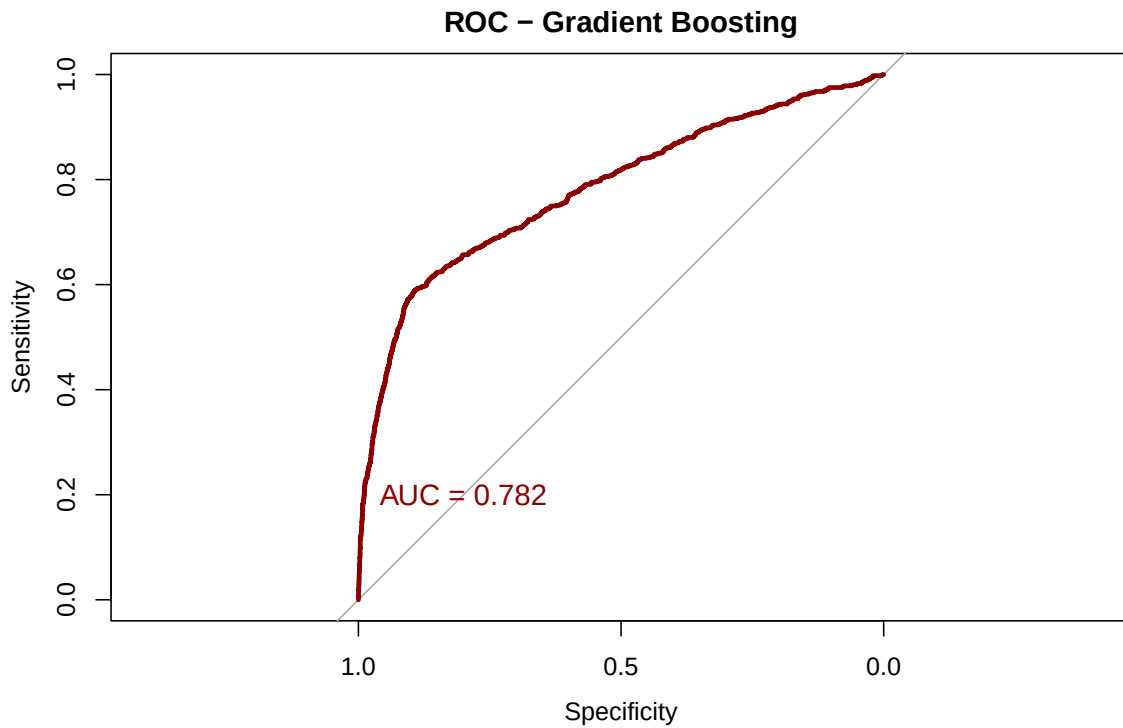
#> AUC Gradient Boosting: 0.7823

```

```

plot(roc_gbm, col = "darkred", lwd = 3, main = "ROC - Gradient Boosting")
text(0.8, 0.2, paste("AUC =", round(auc_gbm, 3)), col = "darkred", cex = 1.2)

```



COMPARACIÓN FORMAL DE VALIDACIÓN CRUZADA (re-samples)

*# resamples() junta los resultados de validación cruzada de varios modelos
entrenados con el MISMO trainControl (mismos folds exactos), permitiendo
comparar no solo el promedio de ROC, sino también la variabilidad entre
folds de cada modelo. El baseline no se incluye aquí porque no es un
objeto train() con resamples de CV.*

```
comparacion_cv <- resamples(list(
  Logistica    = glm_fit,
  KNN          = knn_fit,
  SVM_Lineal   = svm_lineal,
  SVM_RBF      = svm_rbf,
  Arbol        = arbol_fit,
  RandomForest = rf_fit,
  Boosting     = gbm_fit
))
```

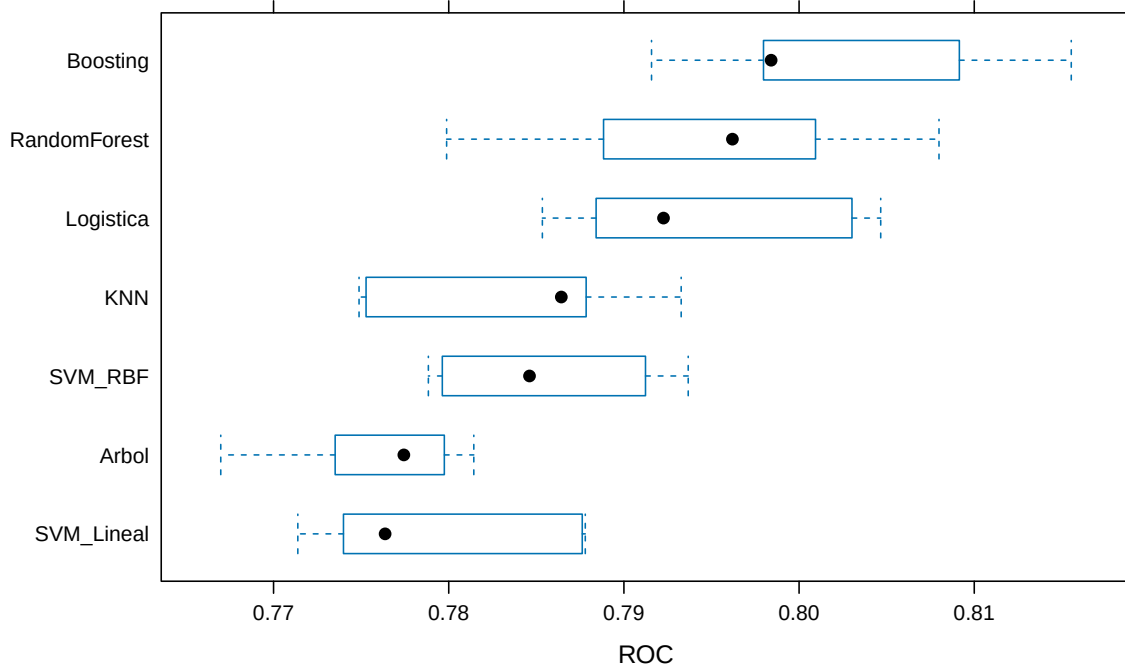
```
summary(comparacion_cv)
```

```
#>
```

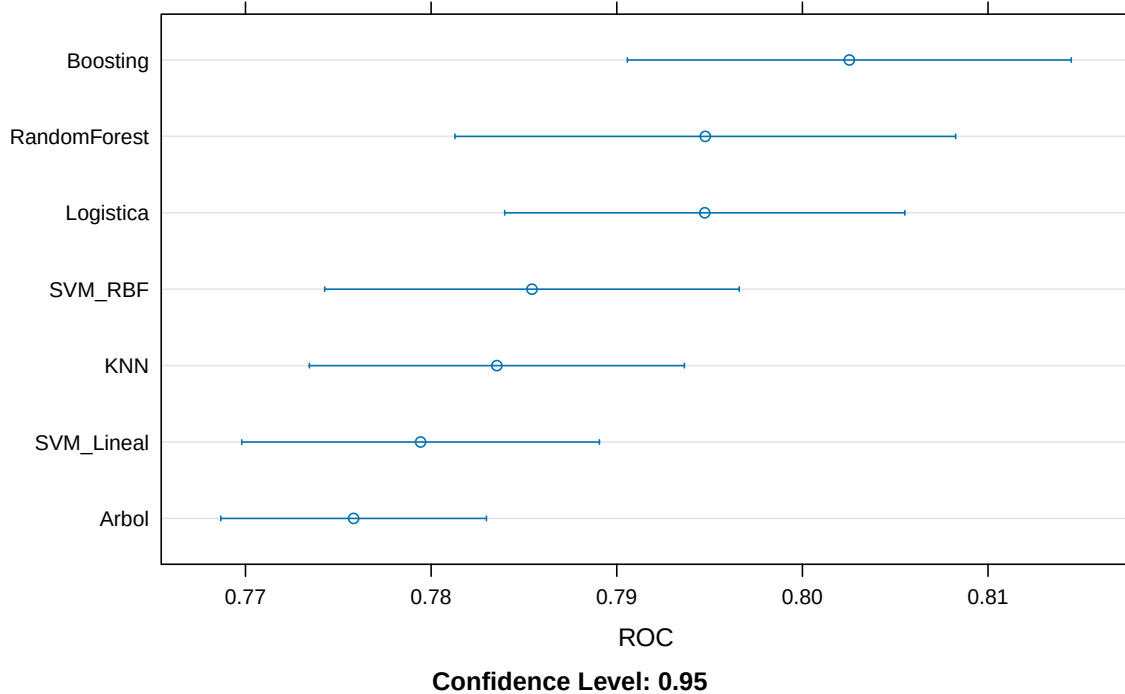
```
#> Call:
#> summary.resamples(object = comparacion_cv)
#>
#> Models: Logistica, KNN, SVM_Lineal, SVM_RBF, Arbol, RandomForest, Boosting
#> Number of resamples: 5
#>
#> ROC
#>
#>      Min. 1st Qu. Median   Mean 3rd Qu.   Max. NA's
#> Logistica  0.7853  0.7884 0.7923 0.7947  0.8030 0.8047    0
#> KNN        0.7749  0.7753 0.7864 0.7835  0.7878 0.7933    0
#> SVM_Lineal 0.7714  0.7740 0.7764 0.7794  0.7876 0.7878    0
#> SVM_RBF    0.7788  0.7800 0.7846 0.7854  0.7900 0.7937    1
#> Arbol      0.7670  0.7735 0.7774 0.7758  0.7797 0.7814    0
#> RandomForest 0.7799  0.7888 0.7962 0.7948  0.8009 0.8080    0
#> Boosting   0.7916  0.7980 0.7984 0.8025  0.8092 0.8155    0
#>
#> Sens
#>
#>      Min. 1st Qu. Median   Mean 3rd Qu.   Max. NA's
#> Logistica  0.8330  0.8426 0.8428 0.8428  0.8438 0.8517    0
#> KNN        0.8255  0.8303 0.8334 0.8337  0.8344 0.8450    0
#> SVM_Lineal 0.8570  0.8584 0.8601 0.8606  0.8621 0.8655    0
#> SVM_RBF    0.8024  0.8203 0.8293 0.8242  0.8333 0.8360    1
#> Arbol      0.8411  0.8445 0.8445 0.8512  0.8454 0.8804    0
#> RandomForest 0.8495  0.8507 0.8661 0.8607  0.8679 0.8695    0
#> Boosting   0.8312  0.8354 0.8484 0.8440  0.8495 0.8553    0
#>
#> Spec
#>
#>      Min. 1st Qu. Median   Mean 3rd Qu.   Max. NA's
#> Logistica  0.6307  0.6380 0.6420 0.6441  0.6469 0.6631    0
#> KNN        0.6173  0.6191 0.6312 0.6307  0.6334 0.6523    0
#> SVM_Lineal 0.5984  0.6137 0.6218 0.6218  0.6294 0.6456    0
#> SVM_RBF    0.6474  0.6511 0.6557 0.6639  0.6685 0.6968    1
#> Arbol      0.6043  0.6420 0.6496 0.6485  0.6671 0.6792    0
#> RandomForest 0.6299  0.6375 0.6406 0.6398  0.6429 0.6482    0
#> Boosting   0.6393  0.6523 0.6555 0.6576  0.6563 0.6846    0
```

comparación gráfica

```
# bwplot() muestra un boxplot por modelo, con la distribución del ROC
# entre los 5 folds de cada uno - no solo el promedio puntual, sino
# también qué tan estable/variable fue cada modelo entre folds.
bwplot(comparacion_cv, metric = "ROC")
```



*# dotplot() muestra el promedio con su intervalo de confianza, más fácil
de leer para una tabla comparativa directa.*
`dotplot(comparacion_cv, metric = "ROC")`



PR-AUC Y LOG-LOSS EN VALIDACIÓN CRUZADA (SIN USAR TEST)

```
calcular_metricas_validacion <- function(modelo, nombre) {  
  preds <- modelo$pred  
  
  fg <- preds$yes[preds$obs == "yes"]  
  bg <- preds$yes[preds$obs == "no"]  
  
  pr <- pr.curve(scores.class0 = fg, scores.class1 = bg, curve = FALSE)  
  
  obs_binario <- ifelse(preds$obs == "yes", 1, 0)  
  logloss <- LogLoss(y_pred = preds$yes, y_true = obs_binario)  
  
  data.frame(  
    Modelo = nombre,  
    PR_AUC = pr$auc.integral,  
    LogLoss = logloss  
  )  
}  
  
tabla_pr_logloss <- rbind(  
  calcular_metricas_validacion(glm_fit, "Regresión Logística"),  
  calcular_metricas_validacion(rf_fit, "Random Forest"),  
  calcular_metricas_validacion(gbm_fit, "Gradient Boosting")  
)  
  
print(tabla_pr_logloss)
```

```
#>           Modelo PR_AUC LogLoss  
#> 1 Regresión Logística 0.4412  0.5201  
#> 2      Random Forest 0.4323  0.4470  
#> 3 Gradient Boosting 0.4527  0.5135
```

CALIBRACIÓN DE PROBABILIDADES (SIN USAR TEST)

```
# Ordenamos las predicciones de cada modelo por rowIndex (la posición  
# original en train), para que las tres columnas queden alineadas
```

```

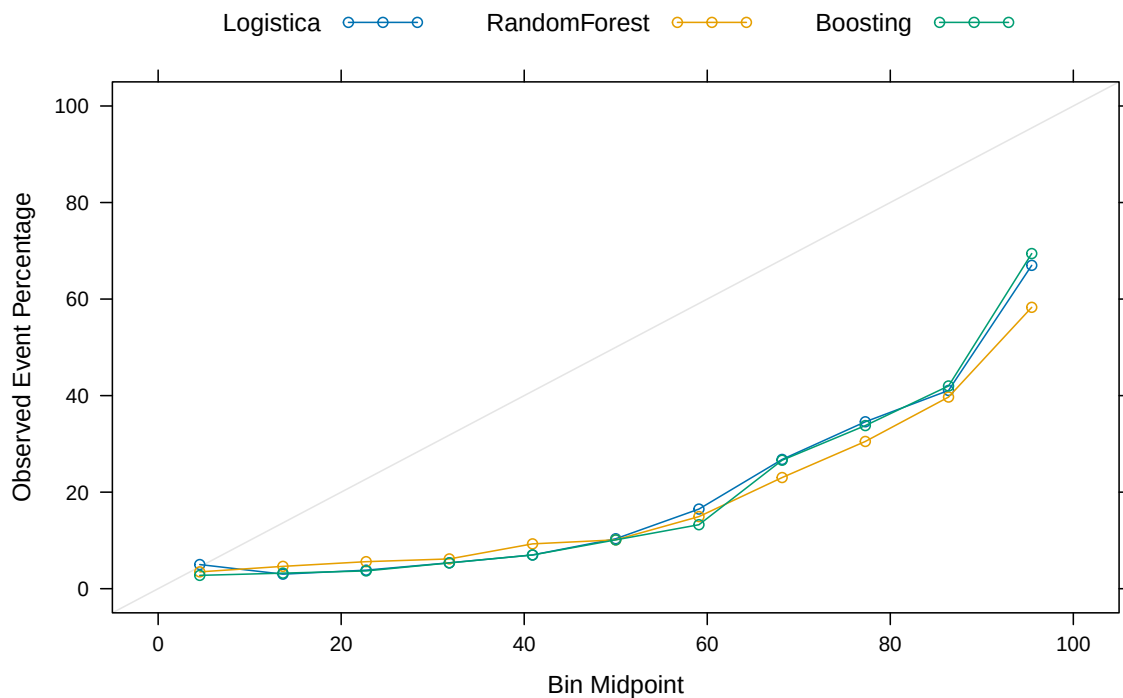
# observación por observación. Como los 3 modelos comparten el mismo
# "control" (mismos folds), esto es válido.
pred_logit_ord <- glm_fit$pred[order(glm_fit$pred$rowIndex), ]
pred_rf_ord    <- rf_fit$pred[order(rf_fit$pred$rowIndex), ]
pred_gbm_ord   <- gbm_fit$pred[order(gbm_fit$pred$rowIndex), ]

# Armos un solo data.frame: una columna "obs" (la clase real) y una
# columna de probabilidad de "yes" por modelo.
calib_df <- data.frame(
  obs      = pred_logit_ord$obs,
  Logistica = pred_logit_ord$yes,
  RandomForest = pred_rf_ord$yes,
  Boosting  = pred_gbm_ord$yes
)

# calibration() agrupa las probabilidades en bins (por defecto deciles) y
# compara, dentro de cada bin, la proporción REAL de "yes" contra la
# probabilidad promedio que el modelo predijo ahí. Un modelo perfectamente
# calibrado cae sobre la diagonal punteada del gráfico.
cal_obj <- calibration(
  obs ~ Logistica + RandomForest + Boosting,
  data = calib_df,
  class = "yes"
)

plot(cal_obj, auto.key = list(columns = 3))

```



UMBRAL ÓPTIMO SEGÚN ÍNDICE DE YODEN (EN VALIDACIÓN, NO EN TEST)

```
# El índice de Youden identifica el punto de la curva ROC que maximiza
# Sensibilidad + Especificidad - 1, es decir, el punto más alejado de la
# diagonal de clasificación aleatoria. A diferencia del umbral de costos
# (que exige que la probabilidad sea "real"), Youden solo necesita que el
# modelo ORDENE bien a las observaciones - por eso es válido usarlo aunque
# las probabilidades estén mal calibradas por el down-sampling.
#
# Calculamos la ROC de validación cruzada con las predicciones out-of-fold
# que caret ya guardó ($pred), NO con test.

roc_logit_cv <- roc(
  response = pred_logit_ord$obs,
  predictor = pred_logit_ord$yes,
  levels = c("no", "yes")
)

roc_rf_cv <- roc(
  response = pred_rf_ord$obs,
  predictor = pred_rf_ord$yes,
  levels = c("no", "yes")
)

roc_gbm_cv <- roc(
  response = pred_gbm_ord$obs,
  predictor = pred_gbm_ord$yes,
  levels = c("no", "yes")
)

youden_logit <- coords(
  roc_logit_cv,
  x = "best",
  best.method = "youden",
  ret = c("threshold", "sensitivity", "specificity")
)

youden_rf <- coords(
  roc_rf_cv,
  x = "best",
  best.method = "youden",
  ret = c("threshold", "sensitivity", "specificity")
)

youden_gbm <- coords(
  roc_gbm_cv,
```

```
x = "best",
best.method = "youden",
ret = c("threshold", "sensitivity", "specificity")
)
```

```
cat("=== Umbral Youden - Regresión Logística ===\n")
```

```
#> === Umbral Youden - Regresión Logística ===
```

```
print(youden_logit)
```

```
#>   threshold sensitivity specificity
#> 1      0.5312      0.6272      0.8639
```

```
cat("\n=== Umbral Youden - Random Forest ===\n")
```

```
#>
```

```
#> === Umbral Youden - Random Forest ===
```

```
print(youden_rf)
```

```
#>   threshold sensitivity specificity
#> 1      0.5583      0.6255      0.8772
```

```
cat("\n=== Umbral Youden - Gradient Boosting ===\n")
```

```
#>
```

```
#> === Umbral Youden - Gradient Boosting ===
```

```
print(youden_gbm)
```

```
#>   threshold sensitivity specificity
#> 1      0.4874      0.6684      0.8361
```

```
umbral final
```

```
umbral_final <- as.numeric(youden_gbm["threshold"])
```

=====

DEFINICION DE COSTOS Y FUNCION AUXILIAR (se definen aca, ANTES de usarse

en el analisis de sensibilidad y en la comparacion Youden vs. Costos)

=====

```

# CF_P (falso positivo): costo de una llamada sin resultado.
# CF_N (falso negativo): costo de una venta potencial perdida.
# Supuesto: el valor de una venta perdida es ~5 veces el costo de una
# llamada sin resultado.
costo_fp <- 1
costo_fn <- 5

# Funcion auxiliar para calcular el costo total dado un vector de
# predicciones y las clases reales:
calcular_costo <- function(pred, real, costo_fp, costo_fn) {
  cm <- confusionMatrix(pred, real, positive = "yes")
  tabla <- cm$table
  fp <- tabla["yes", "no"]
  fn <- tabla["no", "yes"]
  fp * costo_fp + fn * costo_fn
}

```

===== **SENSIBILIDAD DEL UMBRAL DE COSTOS A LOS SUPUES-** **TOS DE CF_N/CF_P** =====

```

razones_prueba <- c(2, 3, 5, 8, 10)

sensibilidad_costos <- data.frame()
for (razon in razones_prueba) {
  cfn_prueba <- razon * costo_fp
  umbral_prueba <- costo_fp / (costo_fp + cfn_prueba)

  pred_prueba <- factor(
    ifelse(pred_gbm_ord$yes >= umbral_prueba, "yes", "no"),
    levels = c("no", "yes")
  )

  costo_total <- calcular_costo(pred_prueba, pred_gbm_ord$obs, costo_fp, cfn_prueba)

  sensibilidad_costos <- rbind(sensibilidad_costos, data.frame(
    Razon_CFN_CFP = razon,
    Umbral = umbral_prueba,
    Costo_total = costo_total
  ))
}

print(sensibilidad_costos)

```

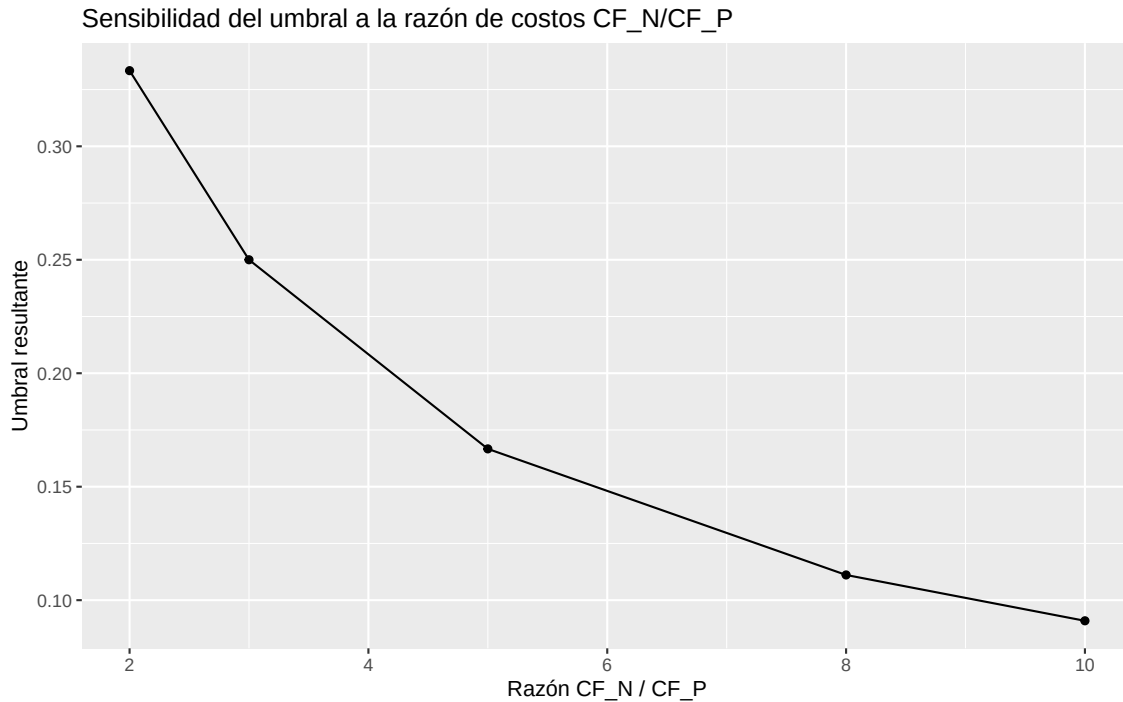
```

#>   Razon_CFN_CFP  Umbral Costo_total

```

```
#> 1          2 0.33333      13201
#> 2          3 0.25000      20254
#> 3          5 0.16667      27493
#> 4          8 0.11111      29046
#> 5         10 0.09091      29129
```

```
ggplot(sensibilidad_costos, aes(x = Razon_CFN_CFP, y = Umbral)) +
  geom_line() + geom_point() +
  labs(title = "Sensibilidad del umbral a la razón de costos CF_N/CF_P",
       x = "Razón CF_N / CF_P", y = "Umbral resultante")
```



COMPARACIÓN: UMBRAL DE YAUDEN vs. UMBRAL DE COSTOS (EN VALIDACIÓN)

```
# =====
# COMPARACIÓN: UMBRAL DE YAUDEN vs. UMBRAL DE COSTOS (EN VALIDACIÓN)
# =====

# Definimos los costos asumidos para cada tipo de error, como se explicó
# en la Sección 2 (Problema y objetivos):
# - CF_P (falso positivo): costo de una llamada sin resultado.
# - CF_N (falso negativo): costo de una venta potencial perdida.
```

```
# Supuesto: el valor de una venta perdida es ~5 veces el costo de una llamada sin resultado.
```

```
costo_fp <- 1
costo_fn <- 5
```

```
# Umbral que minimiza el costo esperado, según la regla de Bayes con costos vista en clase:  $c^* = CF_P / (CF_P + CF_N)$ 
umbral_costos <- costo_fp / (costo_fp + costo_fn)
cat("Umbral basado en costos:", round(umbral_costos, 4), "\n")
```

```
#> Umbral basado en costos: 0.1667
```

```
# Función auxiliar para calcular el costo total dado un vector de predicciones y las clases reales:
```

```
calcular_costo <- function(pred, real, costo_fp, costo_fn) {
  cm <- confusionMatrix(pred, real, positive = "yes")
  tabla <- cm$table
  fp <- tabla["yes", "no"]
  fn <- tabla["no", "yes"]
  fp * costo_fp + fn * costo_fn
}
```

```
# Comparamos ambos criterios SOLO con las predicciones de validación cruzada de Boosting (pred_gbm_ord), sin tocar test todavía.
```

```
pred_gbm_youden_cv <- factor(
  ifelse(pred_gbm_ord$yes >= umbral_final, "yes", "no"),
  levels = c("no", "yes")
)
```

```
pred_gbm_costos_cv <- factor(
  ifelse(pred_gbm_ord$yes >= umbral_costos, "yes", "no"),
  levels = c("no", "yes")
)
```

```
comparacion_umbrales <- data.frame(
  Criterio = c("Youden", "Costos"),
  Umbral = c(umbral_final, umbral_costos),
  Costo_total_CV = c(
    calcular_costo(pred_gbm_youden_cv, pred_gbm_ord$obs, costo_fp, costo_fn),
    calcular_costo(pred_gbm_costos_cv, pred_gbm_ord$obs, costo_fp, costo_fn)
  )
)
```

```
print(comparacion_umbrales)
```

```
#>   Criterio Umbral Costo_total_CV
#> 1   Youden 0.4874         10947
#> 2    Costos 0.1667         27493
```

El umbral basado en costos (0.1667) resulta en un costo total mayor (27,493) que el umbral de Youden (0.4874, costo 10,947), a pesar de que la fórmula de costos es teóricamente óptima. Esto se debe a que dicha fórmula asume probabilidades bien calibradas, mientras que el gráfico de calibración (Sección

10) mostró que Gradient Boosting sobre-estima sistemáticamente la probabilidad de “yes” por efecto del down-sampling dentro de cada fold. El umbral de Youden, al depender solo del ranking relativo (no del valor absoluto de la probabilidad), resulta más robusto ante esta falta de calibración. Por este motivo, se selecciona el umbral de Youden (0.4874) como punto de operación del sistema final. #

=====

SISTEMA FINAL CONGELADO: GRADIENT BOOSTING + UMBRAL DE YODEN

=====

```
# El sistema queda congelado con: algoritmo = Gradient Boosting (gbm_fit,
# con los hiperparámetros ya elegidos por validación cruzada), y umbral =
# umbral_final (Youden, 0.4874), seleccionado en la sección anterior SIN
# haber mirado test. A partir de aquí, "test" se abre UNA sola vez.
```

```
cat("=== SISTEMA FINAL CONGELADO ===\n")
```

```
#> === SISTEMA FINAL CONGELADO ===
```

```
cat("Modelo: Gradient Boosting\n")
```

```
#> Modelo: Gradient Boosting
```

```
cat("Umbral:", round(umbral_final, 4), "(índice de Youden, elegido en validación)\n\n")
```

```
#> Umbral: 0.4874 (índice de Youden, elegido en validación)
```

```
# prob_gbm ya se calculó antes (predict de gbm_fit sobre test). Aplicamos
# el umbral final, en vez del 0.5 usado en la comparación preliminar.
```

```
pred_final <- factor(
  ifelse(prob_gbm >= umbral_final, "yes", "no"),
  levels = c("no", "yes")
)
```

```
cm_final <- confusionMatrix(pred_final, test$y, positive = "yes")
print(cm_final)
```

```
#> Confusion Matrix and Statistics
```

```
#>
```

```
#>           Reference
```

```
#> Prediction  no  yes
```

```
#>           no 6219 351
```

```
#>           yes 1088 576
```

```
#>
```

```
#>           Accuracy : 0.825
```

```
#>           95% CI : (0.817, 0.833)
```

```
#>           No Information Rate : 0.887
```

```
#>           P-Value [Acc > NIR] : 1
```

```
#>
```

```
#>           Kappa : 0.351
```

```
#>
```

```
#>           McNemar's Test P-Value : <2e-16
```

```
#>
```

```
#>          Sensitivity : 0.621
#>          Specificity : 0.851
#>          Pos Pred Value : 0.346
#>          Neg Pred Value : 0.947
#>          Prevalence : 0.113
#>          Detection Rate : 0.070
#>          Detection Prevalence : 0.202
#>          Balanced Accuracy : 0.736
#>
#>          'Positive' Class : yes
#>
```

```
cat("\n=== RESULTADO DEL SISTEMA FINAL (TEST, UNA SOLA VEZ) ===\n")
```

```
#>
#> === RESULTADO DEL SISTEMA FINAL (TEST, UNA SOLA VEZ) ===
```

```
cat("Accuracy:", round(cm_final$overall["Accuracy"], 4), "\n")
```

```
#> Accuracy: 0.8252
```

```
cat("Sensibilidad:", round(cm_final$byClass["Sensitivity"], 4), "\n")
```

```
#> Sensibilidad: 0.6214
```

```
cat("Especificidad:", round(cm_final$byClass["Specificity"], 4), "\n")
```

```
#> Especificidad: 0.8511
```

```
cat("Precisión:", round(cm_final$byClass["Precision"], 4), "\n")
```

```
#> Precisión: 0.3462
```

```
cat("F1:", round(cm_final$byClass["F1"], 4), "\n")
```

```
#> F1: 0.4446
```

```
# Número de acciones: cuántas llamadas haría el banco según este sistema
```

```
n_llamadas <- sum(pred_final == "yes")
```

```
cat("\nNúmero de acciones (llamadas a realizar):", n_llamadas, "de", length(pred_final), "clientes en test")
```

```
#>
```

```
#> Número de acciones (llamadas a realizar): 1664 de 8234 clientes en test
```

```
# Costo total en TEST (una sola vez, con el sistema ya congelado):
```

```
costo_total_test <- calcular_costo(pred_final, test$y, costo_fp, costo_fn)
```

```
cat("Costo total en test:", costo_total_test, "\n")
```

```
#> Costo total en test: 2843
```

COMPARACIÓN: SISTEMA FINAL vs. BASELINE (EN TEST)

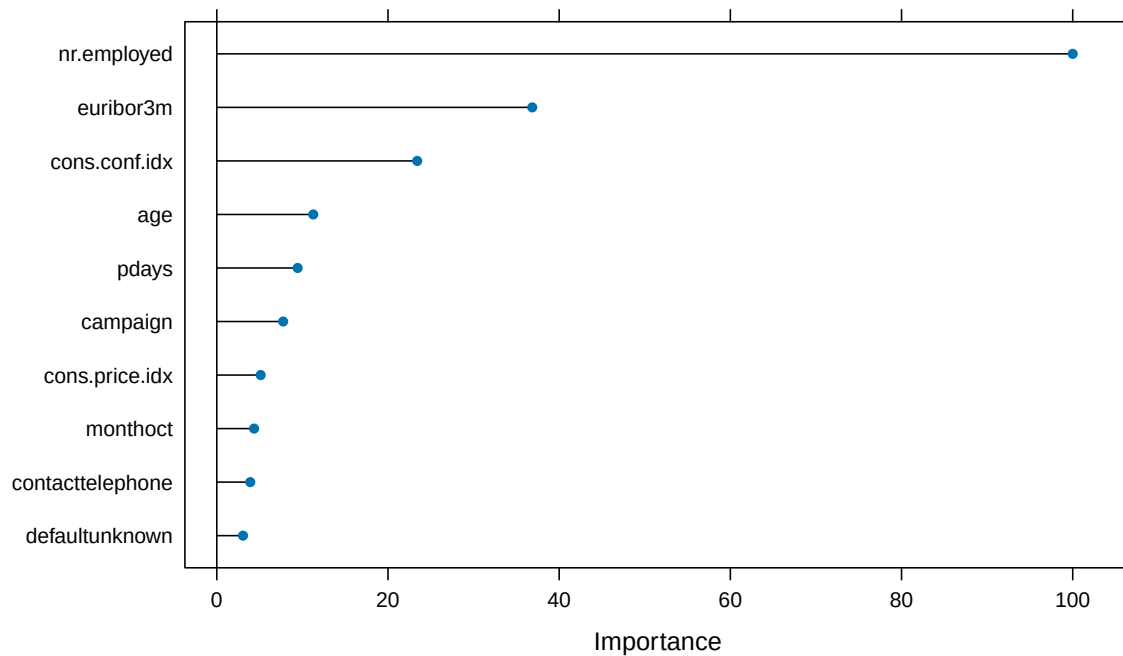
```
tabla_final_vs_baseline <- data.frame(  
  Sistema = c("Baseline (siempre 'no')", "Gradient Boosting (umbral Youden)"),  
  Accuracy = c(cm_baseline$overall["Accuracy"], cm_final$overall["Accuracy"]),  
  Sensibilidad = c(cm_baseline$byClass["Sensitivity"], cm_final$byClass["Sensitivity"]),  
  Especificidad = c(cm_baseline$byClass["Specificity"], cm_final$byClass["Specificity"]),  
  Numero_acciones = c(0, n_llamadas)  
)  
  
print(tabla_final_vs_baseline)
```

```
#>               Sistema Accuracy Sensibilidad Especificidad  
#> 1      Baseline (siempre 'no')  0.8874      0.0000      1.0000  
#> 2 Gradient Boosting (umbral Youden)  0.8252      0.6214      0.8511  
#> Numero_acciones  
#> 1              0  
#> 2            1664
```

INTERPRETACIÓN: IMPORTANCIA DE VARIABLES EN EL MODELO FINAL (BOOSTING)

```
# Igual que con Random Forest, varImp() funciona con cualquier modelo de  
# caret, incluyendo gbm. Mide qué tanto contribuye cada variable a reducir  
# el error del modelo en promedio, a través de los árboles secuenciales.  
# RECORDATORIO IMPORTANTE: esto indica utilidad predictiva dentro del  
# modelo, NO causalidad - una variable importante no significa que  
# cambiarla artificialmente cambiaría el comportamiento real del cliente.  
#install.packages("gbm")  
library(gbm)  
varImp(gbm_fit) %>%  
  plot(top = 10, main = "Importancia de variables - Gradient Boosting (modelo final)")
```

Importancia de variables – Gradient Boosting (modelo final)



```
# También como tabla, para citar valores exactos en el texto del informe:
importancia_gbm <- varImp(gbm_fit)$importance
importancia_gbm <- importancia_gbm[order(-importancia_gbm$Overall), , drop = FALSE]
head(importancia_gbm, 10)
```

```
#>          Overall
#> nr.employed 100.000
#> euribor3m   36.853
#> cons.conf.idx 23.415
#> age         11.272
#> pdays        9.443
#> campaign     7.754
#> cons.price.idx 5.125
#> monthoct     4.359
#> contacttelephone 3.907
#> defaultunknown 3.060
```

CARACTERIZACIÓN DE FALSOS POSITIVOS Y FALSOS NEGATIVOS

```
# Armamos un data.frame con el resultado de cada cliente de test: si fue
# TP, TN, FP o FN, según la predicción final vs. la clase real.
```

```
analisis_errores <- test %>%
  mutate(
    prediccion = pred_final,
    resultado = case_when(
      y == "yes" & prediccion == "yes" ~ "Verdadero Positivo",
      y == "no" & prediccion == "no" ~ "Verdadero Negativo",
      y == "no" & prediccion == "yes" ~ "Falso Positivo",
      y == "yes" & prediccion == "no" ~ "Falso Negativo"
    )
  )
```

```
# Comparamos el perfil promedio de los Falsos Negativos (clientes que sí
# aceptaban pero el modelo no detectó) contra los Verdaderos Positivos
# (los que sí detectó), para ver qué los distingue:
```

```
analisis_errores %>%
  filter(resultado %in% c("Falso Negativo", "Verdadero Positivo")) %>%
  group_by(resultado) %>%
  summarise(
    edad_promedio = mean(age),
    contactos_previos_promedio = mean(previous),
    campaign_promedio = mean(campaign),
    pct_poutcome_success = mean(poutcome == "success"),
    .groups = "drop"
  )
```

```
#> # A tibble: 2 x 5
#>   resultado      edad_promedio contactos_previos_promedio campaign_promedio
#>   <chr>          <dbl>                <dbl>                <dbl>
#> 1 Falso Negativo      39.4                0.0997                2.58
#> 2 Verdadero Positivo  42.1                0.792                1.76
#> # i 1 more variable: pct_poutcome_success <dbl>
```

```
# Mismo análisis para Falsos Positivos vs Verdaderos Negativos:
```

```
analisis_errores %>%
  filter(resultado %in% c("Falso Positivo", "Verdadero Negativo")) %>%
  group_by(resultado) %>%
  summarise(
    edad_promedio = mean(age),
    contactos_previos_promedio = mean(previous),
    campaign_promedio = mean(campaign),
    pct_poutcome_success = mean(poutcome == "success"),
    .groups = "drop"
  )
```

```
#> # A tibble: 2 x 5
#>   resultado      edad_promedio contactos_previos_promedio campaign_promedio
#>   <chr>          <dbl>                <dbl>                <dbl>
#> 1 Falso Positivo      40.5                0.374                2.05
#> 2 Verdadero Negativo  39.9                0.0868                2.80
#> # i 1 more variable: pct_poutcome_success <dbl>
```

DEFINICIÓN DE COSTOS (se usa desde acá en adelante, incluida la tabla

comparativa de la Sección 9 y el umbral de costos de la Sección 11)

```
# =====  
# DEFINICIÓN DE COSTOS Y FUNCIÓN AUXILIAR (se usan desde acá en adelante,  
# incluida la tabla comparativa de la Sección 9 y el umbral de costos de  
# la Sección 11)  
# =====  
costo_fp <- 1  
costo_fn <- 5  
  
# Función auxiliar para calcular el costo total dado un vector de  
# predicciones y las clases reales:  
calcular_costo <- function(pred, real, costo_fp, costo_fn) {  
  cm <- confusionMatrix(pred, real, positive = "yes")  
  tabla <- cm$table  
  fp <- tabla["yes", "no"]  
  fn <- tabla["no", "yes"]  
  fp * costo_fp + fn * costo_fn  
}
```

TABLA COMPARATIVA COMPLETA (VALIDACIÓN) — LOS 8 MODELOS

```
# Esta función calcula, para UN modelo, las 9 columnas que exige la  
# tabla obligatoria de la pauta (ROC-AUC, PR-AUC, Log-loss, Precision,  
# Recall, Especificidad, F1, Costo, Volumen), usando las predicciones de  
# validación cruzada ($pred) que caret ya guardó automáticamente gracias  
# a savePredictions = "final" en el control. No usa test en ningún  
# momento, tal como exige la Sección 9 (comparar SOLO en validación).  
calcular_fila_completa <- function(modelo, nombre) {  
  preds <- modelo$pred  
  
  # Si el modelo tuvo algún problema numérico en algún fold (le pasó a  
  # SVM RBF), puede haber filas con NA en la probabilidad predicha.
```

```

# Las descartamos antes de calcular las métricas, dejando registro de
# cuántas se sacaron, para no romper pr.curve()/roc()/LogLoss().
n_na <- sum(is.na(preds$yes))
if (n_na > 0) {
  cat("Advertencia:", nombre, "- se descartaron", n_na, "predicciones con NA\n")
  preds <- preds[!is.na(preds$yes), ]
}

# ROC-AUC: área bajo la curva ROC, mide qué tan bien el modelo ordena
# a los clientes de mayor a menor probabilidad de aceptar.
roc_obj <- roc(preds$obs, preds$yes, levels = c("no", "yes"), direction = "<", quiet = TRUE)
roc_auc <- as.numeric(auc(roc_obj))

# PR-AUC: similar a ROC-AUC, pero no se deja engañar por el desbalance
# de clases (~11% "yes"). fg = probabilidades de los "yes" reales
# (foreground), bg = probabilidades de los "no" reales (background).
fg <- preds$yes[preds$obs == "yes"]
bg <- preds$yes[preds$obs == "no"]
pr_auc <- pr.curve(scores.class0 = fg, scores.class1 = bg, curve = FALSE)$auc.integral

# Log-loss: mide qué tan "confiables" son las probabilidades en sí
# mismas (no solo el orden). Penaliza fuerte estar muy seguro y
# equivocarse. Convertimos "yes"/"no" a 1/0 porque LogLoss() lo pide así.
obs_binario <- ifelse(preds$obs == "yes", 1, 0)
logloss <- LogLoss(y_pred = preds$yes, y_true = obs_binario)

# Para Precision/Recall/Especificidad/F1/Costo/Volumen necesitamos una
# clase concreta (no una probabilidad), así que aplicamos el umbral
# tradicional de 0.5 acá - esto es solo para que los 8 modelos se
# comparen en el MISMO punto operacional; el umbral final de Youden se
# aplica más adelante, solo al modelo elegido.
pred_05 <- factor(ifelse(preds$yes >= 0.5, "yes", "no"), levels = c("no", "yes"))
cm <- confusionMatrix(pred_05, preds$obs, positive = "yes")

# calcular_costo() ya la definimos antes (en el bloque de umbral de
# costos): fp*costo_fp + fn*costo_fn. Volumen = cuántas veces predijo "yes".
costo <- calcular_costo(pred_05, preds$obs, costo_fp, costo_fn)
volumen <- sum(pred_05 == "yes")

# unname() es importante: sin esto, cm$byClass["Precision"] arrastra el
# texto "Precision" como nombre, y ese nombre termina apareciendo como
# nombre de FILA al juntar todo con rbind() más abajo, dejando la tabla
# con etiquetas feas en vez de números de fila limpios.
data.frame(
  Modelo = nombre,
  ROC_AUC = roc_auc,
  PR_AUC = pr_auc,
  Log_loss = logloss,
  Precision = unname(cm$byClass["Precision"]),
  Recall = unname(cm$byClass["Sensitivity"]),
  Especificidad = unname(cm$byClass["Specificity"]),
  F1 = unname(cm$byClass["F1"]),
  Costo = costo,

```

```

    Volumen = volumen
  )
}

# El baseline se calcula APARTE, con sus propios valores, porque no es un
# objeto train() con validación cruzada - no tiene $pred, así que no
# puede pasar por calcular_fila_completa(). ROC-AUC/PR-AUC/Log-loss/
# Precision quedan como NA porque el baseline nunca predice "yes", por lo
# que esas métricas no se pueden calcular de forma significativa para él.
fila_baseline <- data.frame(
  Modelo = "Baseline",
  ROC_AUC = NA,
  PR_AUC = NA,
  Log_loss = NA,
  Precision = NA,
  Recall = unname(cm_baseline$byClass["Sensitivity"]),
  Especificidad = unname(cm_baseline$byClass["Specificity"]),
  F1 = unname(cm_baseline$byClass["F1"]),
  Costo = calcular_costo(pred_baseline, test$y, costo_fp, costo_fn),
  Volumen = 0
)

# rbind() apila todas las filas (una por modelo) en una sola tabla,
# llamando a calcular_fila_completa() una vez por cada uno de los 7
# modelos con predictores, más la fila del baseline ya calculada aparte.
tabla_comparativa_completa <- rbind(
  fila_baseline,
  calcular_fila_completa(glm_fit, "Regresión Logística"),
  calcular_fila_completa(knn_fit, "KNN"),
  calcular_fila_completa(svm_lineal, "SVM Lineal"),
  calcular_fila_completa(svm_rbf, "SVM RBF"),
  calcular_fila_completa(arbol_fit, "Árbol"),
  calcular_fila_completa(rf_fit, "Random Forest"),
  calcular_fila_completa(gbm_fit, "Gradient Boosting")
)

```

#> Advertencia: SVM RBF - se descartaron 13178 predicciones con NA

```

# Ordenamos de mayor a menor ROC-AUC, para que el mejor modelo quede
# arriba y sea fácil de identificar de un vistazo.
tabla_comparativa_completa <- tabla_comparativa_completa[order(-tabla_comparativa_completa$ROC_AUC), ]

# rownames(...) <- NULL limpia los números/etiquetas de fila que R deja
# después del rbind()+order() (por ejemplo "3", "7", "Precision6"),
# dejando la tabla con una numeración simple y ordenada al imprimirla.
rownames(tabla_comparativa_completa) <- NULL

# Columna "Subgrupo": la pauta la exige en la tabla, pero según la
# Rúbrica (dimensión "Equidad y subgrupos"), el análisis detallado de
# subgrupo se pide sobre el SISTEMA FINAL, no repetido para los 8
# modelos candidatos. Por eso esta columna es solo una referencia de
# dónde está ese análisis (Sección 13), no un número recalculado aquí.
tabla_comparativa_completa$Subgrupo <- "Ver Sección 13 (detalle por mes, modelo final)"

```

```
print(tabla_comparativa_completa)
```

```
#>           Modelo ROC_AUC PR_AUC Log_loss Precision Recall Especificidad
#> 1 Gradient Boosting 0.8023 0.4527 0.5135 0.3486 0.6576 0.8440
#> 2 Regresión Logística 0.7946 0.4412 0.5201 0.3423 0.6441 0.8428
#> 3 Random Forest 0.7945 0.4323 0.4470 0.3672 0.6401 0.8599
#> 4 SVM RBF 0.7850 0.4024 0.5259 0.3241 0.6639 0.8242
#> 5 KNN 0.7835 0.4035 0.5323 0.3249 0.6315 0.8334
#> 6 Árbol 0.7765 0.3677 0.5445 0.3562 0.6484 0.8512
#> 7 SVM Lineal 0.7745 0.3989 0.5331 0.3616 0.6218 0.8606
#> 8 Baseline NA NA NA NA 0.0000 1.0000
#>           F1 Costo Volumen Subgrupo
#> 1 0.4557 10916 7002 Ver Sección 13 (detalle por mes, modelo final)
#> 2 0.4470 11200 6986 Ver Sección 13 (detalle por mes, modelo final)
#> 3 0.4667 10774 6470 Ver Sección 13 (detalle por mes, modelo final)
#> 4 0.4356 9100 6081 Ver Sección 13 (detalle por mes, modelo final)
#> 5 0.4291 11710 7214 Ver Sección 13 (detalle por mes, modelo final)
#> 6 0.4598 10875 6757 Ver Sección 13 (detalle por mes, modelo final)
#> 7 0.4573 11094 6382 Ver Sección 13 (detalle por mes, modelo final)
#> 8 NA 4635 0 Ver Sección 13 (detalle por mes, modelo final)
```

```
# Versión con formato más limpio (tabla en vez de texto plano), lista
# para que se vea bien al generar el HTML final del informe:
knitr::kable(tabla_comparativa_completa, digits = 4,
              caption = "Tabla comparativa de validación cruzada (los 8 modelos)")
```

Cuadro 1: Tabla comparativa de validación cruzada (los 8 modelos)

Modelo	ROC_AUC	PR_AUC	Log_loss	Precision	Recall	Especificidad	F1	Costo	Volumen	Subgrupo
Gradient Boosting	0.8023	0.4527	0.5135	0.3486	0.6576	0.8440	0.4557	10916	7002	Ver Sección 13 (detalle por mes, modelo final)
Regresión Logística	0.7946	0.4412	0.5201	0.3423	0.6441	0.8428	0.4470	11200	6986	Ver Sección 13 (detalle por mes, modelo final)
Random Forest	0.7945	0.4323	0.4470	0.3672	0.6401	0.8599	0.4667	10774	6470	Ver Sección 13 (detalle por mes, modelo final)
SVM RBF	0.7850	0.4024	0.5259	0.3241	0.6639	0.8242	0.4356	9100	6081	Ver Sección 13 (detalle por mes, modelo final)
KNN	0.7835	0.4035	0.5323	0.3249	0.6315	0.8334	0.4291	11710	7214	Ver Sección 13 (detalle por mes, modelo final)
Árbol	0.7765	0.3677	0.5445	0.3562	0.6484	0.8512	0.4598	10875	6757	Ver Sección 13 (detalle por mes, modelo final)
SVM Lineal	0.7745	0.3989	0.5331	0.3616	0.6218	0.8606	0.4573	11094	6382	Ver Sección 13 (detalle por mes, modelo final)
Baseline	NA	NA	NA	NA	0.0000	1.0000	NA	4635	0	Ver Sección 13 (detalle por mes, modelo final)